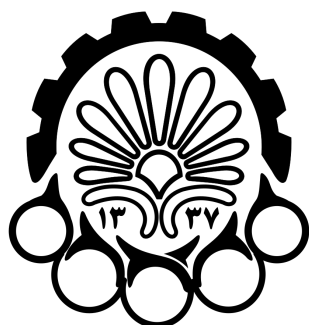




دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

دانشکده مهندسی کامپیوتر

پایان نامه پروژه کارشناسی

عنوان پروژه:

پیاده سازی یک ابزار تحلیل برای زبان برنامه نویسی C
به منظور کاهش مشکلات ایمنی حافظه

نگارنده:

امیرحسین قدیری

استاد راهنما:

مهران سلیمان فلاح

تابستان ۱۴۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

اینجانب امیرحسین قدیری متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آن‌ها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر است. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است.

نقل مطالب با ذکر مآخذ بلامانع است.

امیرحسین قدیری

امضا



سپاس‌گزاری

بدینوسیله مراتب قدردانی و امتنان خود را خدمت جناب آقای دکتر سلیمان فلاح، استاد راهنمای گرانقدر، که راهنمایی‌های خود را در تمام مراحل بر من ارزانی داشتند و بدون کمک‌های ایشان انجام این پروژه امکان‌پذیر نبود، ابراز و از زحمات ایشان بسیار سپاس‌گزار بوده و آرزومند توفیق روزافزون برایشان می‌باشم.

تابستان ۱۴۰۴

تقدیم

تقدیم به پدر و مادر عزیزم، که با مهربانی، صبر و حمایت بی‌دریغ خود همواره پشتوانه و چراغ راه من بوده‌اند. محبت و انگیزه‌ای که از شما دریافت کرده‌ام، همواره مشوق من در مسیر تحصیل و تلاش علمی بوده و بدون همراهی و دلگرمی‌های شما، دستیابی به این مرحله امکان‌پذیر نبوده است.

تابستان ۱۴۰۴

چکیده

مدیریت حافظه در زبان برنامه‌نویسی C یکی از چالش‌های اساسی در توسعه و نگهداری نرم‌افزارهای پایدار و ایمن است. هر چند امروزه کمتر از این زبان برای توسعه نرم‌افزارهای جدید استفاده می‌شود، همچنان کدهای قدیمی نوشته شده با استفاده از آن نیاز به نگهداری داشته و عدم وجود سازوکاری به‌منظور جلوگیری از خطاهای حافظه مشکل‌های بسیاری ایجاد کرده است. در برنامه‌های نوشته شده با استفاده از این زبان، خطاهایی مانند نشتی حافظه و استفاده از حافظه پس از آزادسازی مواردی هستند که به‌طور معمول مشاهده می‌شوند. منشأ چنین خطاهایی تخصیص و آزادسازی حافظه توسط برنامه‌نویس و عدم وجود سازوکاری به‌منظور مدیریت خودکار حافظه در این زبان است.

این خطاها به‌ویژه در پروژه‌های بزرگ اهمیت بیشتری پیدا می‌کنند زیرا پیدا کردن و شناسایی نشتی‌ها و خطاهای حافظه در کدهای گسترده بسیار دشوار بوده و می‌تواند منجر به کاهش کارایی، بروز خطاهای امنیتی و حتی توقف ناگهانی برنامه شود. بنابراین، وجود ابزارهایی که بتوانند شیوه استفاده برنامه از حافظه را واکاوی کرده و خطاهای مرتبط را شناسایی کنند نقش حیاتی در افزایش پایداری و ایمنی نرم‌افزارهای نوشته شده با استفاده از این زبان دارند. استفاده از چنین ابزارهایی در محیط توسعه نیز دارای اهمیت بسزایی است.

در این گزارش، ابتدا مسئله مدیریت حافظه در زبان برنامه‌نویسی C بیان می‌شود. سپس، راه‌حل‌ها و ابزارهای موجود مانند ولگریند و اسموک بررسی می‌شوند. در ادامه، سامانه نوع خطی به‌عنوان راه‌حل پیشنهادی برای تشخیص خطاهای نشتی حافظه، استفاده از حافظه پس از آزادسازی و آزادسازی دوباره حافظه مورد بررسی قرار گرفته و به‌شکل صوری بیان می‌شود. در نهایت ابزاری بر مبنای سامانه نوع خطی پیاده‌سازی شده و درستی و دقت آن ارزیابی و سنجیده می‌شود. باتوجه به نتایج آزمایش‌ها، ابزار درست بوده و دارای دقت قابل‌قبولی است.

واژه‌های کلیدی:

تحلیل ایستا، زبان برنامه‌نویسی C، سامانه نوع خطی، ایمنی زبان، ایمنی حافظه

فهرست مطالب

فصل اول - مقدمه	۱
۱.۱ بیان مسئله	۲
۲.۱ اهداف پروژه	۳
۳.۱ ابزارها و سامانه‌های مشابه	۴
فصل دوم - مفاهیم پایه	۶
۱.۲ نوع‌ها	۷
۲.۲ سامانه‌های نوع	۸
۱.۲.۲ درستی	۹
۲.۲.۲ کامل بودن	۱۰
۳.۲ سامانه نوع زیرساختاری	۱۱
۴.۲ سامانه نوع خطی	۱۲
۵.۲ ایمنی زبان	۱۳
۶.۲ خطاهای حافظه	۱۴
۱.۶.۲ نشتی حافظه	۱۵
۲.۶.۲ استفاده از حافظه پس از آزادسازی	۱۶
۳.۶.۲ آزادسازی دوباره حافظه	۱۷
۷.۲ تحلیل برنامه	۱۸
۱.۷.۲ تحلیل ایستا	۱۹
۲.۷.۲ تحلیل پویا	۱۹
فصل سوم - توصیف صوری سامانه نوع پیشنهادی	۲۰
۱.۳ واریسی نوع	۲۱
۲.۳ نحو	۲۲
۳.۳ قاعده‌های ارزیابی	۲۵

۴.۳ قاعده‌های نوع‌دهی ۲۹

فصل چهارم - پیاده‌سازی و ارزیابی ۳۵

۱.۴ اعمال قوانین نوع خطی ۳۶

۲.۴ الگوریتم پیشنهادی ۴۰

۳.۴ آزمایش‌ها ۴۰

فصل پنجم - جمع‌بندی و پیشنهادها ۴۵

۱.۵ نتیجه‌گیری و جمع‌بندی ۴۶

۲.۵ چشم‌انداز و پیشنهادها ۴۷

فهرست مراجع ۵۳

فهرست شکل‌ها

- شکل ۱.۲ - بد به کار بردن دسترسی به حافظه پس از آزادسازی ۱۶
- شکل ۲.۲ - بد به کار بردن آزادسازی دوباره حافظه ۱۷
- شکل ۱.۳ - توصیف نحو نوع‌ها و مقدارها ۲۲
- شکل ۲.۳ - توصیف نحو عبارت‌ها و عملیات‌ها ۲۳
- شکل ۳.۳ - توصیف نحو حکم‌ها ۲۴
- شکل ۴.۳ - توصیف نحو برنامه ۲۴
- شکل ۵.۳ - توصیف محیط و مجموعه توابع ۲۵
- شکل ۶.۳ - قاعده‌های ارزیابی عبارت‌ها ۲۷
- شکل ۷.۳ - قاعده‌های ارزیابی جمله‌ها ۲۸
- شکل ۸.۳ - قاعده‌های ارزیابی برنامه ۲۹
- شکل ۹.۳ - توصیف زمینه نوع و زمینه متغیرهای خطی ۲۹
- شکل ۱۰.۳ - قاعده‌های نوع‌دهی عملیات‌ها ۳۰
- شکل ۱۱.۳ - قاعده‌های نوع‌دهی عبارت‌ها ۳۲
- شکل ۱۲.۳ - قاعده‌های نوع‌دهی جمله‌ها ۳۳
- شکل ۱۳.۳ - قاعده‌های نوع‌دهی برنامه ۳۴
- شکل ۱.۴ - نمودار بلوکی ابزار ۳۶
- شکل ۲.۴ - قطعه کد ساده دارای شاخه ۳۸
- شکل ۳.۴ - نمایش گراف کنترل جریان برای یک قطعه کد ساده توسط ابزار ۳۸
- شکل ۴.۴ - نمایش مرسوم گراف کنترل جریان برای یک قطعه کد ساده ۳۹
- شکل ۵.۴ - دستورهای استفاده شده به منظور تولید نمونه‌های آزمایشی ۴۱
- شکل ۶.۴ - نمونه برنامه آزمایش شده که دارای ایراد حافظه است ۴۳
- شکل ۷.۴ - نمودار دایره‌ای ارزیابی ابزار بر روی ۲۲۰ نمونه آزمایشی ۴۴
- شکل ۱.۵ - تعریف عمومی ماکروی نوع خطی ۴۷

شکل ۲.۵ - تعریف اختصاصی ماکروی نوع خطی	۴۸
شکل ۳.۵ - نمونه حاشیه‌نویسی کتابخانه استاندارد	۴۸
شکل ۴.۵ - نمونه ساختار فایل دستوره‌های کامپایل	۴۹
شکل ۵.۵ - نمونه استفاده از ماکروها	۵۰
شکل ۶.۵ - نتیجه گسترش ماکروها در بخش پیش‌پردازش	۵۱

فهرست جدول‌ها

- جدول ۱۰۲ - مقایسه درستی با کامل بودن سامانه‌های نوع ۱۰
- جدول ۲۰۲ - انواع سامانه‌های نوع زیرساختاری ۱۲
- جدول ۳۰۲ - مثال‌هایی از زبان‌های ایمن و غیرایمن با توجه به روش‌های بررسی نوع‌ها ۱۳
- جدول ۱۰۴ - نتیجه ارزیابی ابزار بر روی ۲۲۰ نمونه آزمایشی ۴۴

فصل اول

مقدمه

۱.۱ بیان مسئله

مدیریت حافظه در زبان برنامه‌نویسی C یکی از چالش‌های اساسی در توسعه نرم‌افزارهای پایدار و امن به شمار می‌رود. هرچند امروزه این زبان کمتر برای توسعه برنامه‌های حساس استفاده می‌شود اما کدهای قدیمی^۱ نوشته شده با استفاده از این زبان همچنان در بسیاری از پروژه‌های بزرگ وجود دارند. این کدهای قدیمی نیازمند نگهداری و اعمال تغییرهایی هستند و در صورتی که مسائل مربوط به مدیریت حافظه به دقت رعایت نشوند، برنامه‌های نوشته شده با استفاده از این زبان ممکن است مستعد خطاهای جدی شوند.

استفاده از حافظه پس از آزادسازی، نشتی حافظه و آزادسازی دوباره حافظه جزو خطاهای اساسی و پیچیده در زبان‌هایی هستند که ایمنی حافظه^۲ به صورت خودکار تأمین نمی‌شود. در این زبان‌ها، برنامه‌نویس مسئول تخصیص، استفاده و آزادسازی حافظه بوده و هیچ سازوکار خودکاری برای بازیابی حافظه بدون استفاده وجود ندارد. زمانی که برنامه‌نویس حافظه‌ای را آزاد می‌کند ولی همچنان به آن دسترسی دارد، اشاره‌گری معلق^۳ ایجاد می‌شود که می‌تواند موجب رفتارهای غیرقابل پیش‌بینی در برنامه شود. این رفتارها شامل تغییر غیرمنتظره داده‌ها، خراب شدن ساختارهای داده‌ای و در نهایت توقف ناگهانی برنامه هستند. از دیدگاه امنیتی نیز چنین دسترسی‌هایی می‌توانند زمینه‌ساز آسیب‌پذیری‌های جدی شوند. یک مهاجم می‌تواند با بهره‌گیری از حافظه آزاد شده، داده‌های حساس مانند رمزها، اطلاعات شخصی و کلیدهای امنیتی را استخراج کرده و یا آن‌ها را تغییر دهد. این نوع خطاها اغلب در زمان توسعه یا آزمایش برنامه آشکار نمی‌شوند و برنامه حتی با وجود آسیب‌پذیری به کار خود ادامه می‌دهد که این امر تشخیص و پیشگیری از آن‌ها را دشوارتر می‌کند.

برای کاهش خطاهای مرتبط با دسترسی به حافظه آزاد شده، برنامه‌نویسان می‌بایست اصول دقیق مدیریت حافظه را رعایت کنند. از جمله این اصول می‌توان به بررسی مداوم تخصیص‌ها و آزادسازی‌ها، استفاده از ابزارهای تحلیل حافظه برای شناسایی نشتی حافظه و اشاره‌گرهای معلق و طراحی کد به گونه‌ای که دسترسی غیرمجاز به حافظه پس از آزادسازی به حداقل برسد، اشاره نمود. رعایت این نکته‌ها اگرچه می‌تواند به افزایش پایداری، کارایی و ایمنی برنامه کمک کند اما همچنان احتمال بروز خطا وجود داشته و ممکن است برخی موارد از قلم بیافتند. به همین دلیل، نیاز به سازوکاری است که بتواند خطاهای مربوط به حافظه را به طور خودکار شناسایی کند.

از آنجایی که یکی از دلایل اصلی استفاده از زبان برنامه‌نویسی C اهمیت عملکرد و سرعت اجرای برنامه است پس این سازوکار باید به گونه‌ای طراحی شود تا دارای انتزاع بدون هزینه^۴ بوده و هیچگونه اثری در زمان اجرای برنامه از خودش به جا نگذارد و سربار اضافه‌ای نداشته باشد. در غیراین صورت، استفاده از زبان‌های دارای سازوکارهای مدیریت حافظه

^۱ legacy code

^۲ memory safety

^۳ dangling pointer

^۴ zero-cost abstraction

خودکار مانند زباله جمع‌کن‌ها^۵، گزینه مناسب‌تری خواهد بود. استفاده از چنین سازوکاری باعث می‌شود تا برنامه‌نویس بتواند خطاهای ناشی از مدیریت نادرست حافظه را پیش از وقوع مشکل‌های جدی شناسایی و اصلاح کند و در نتیجه ایمنی، پایداری و قابلیت اعتماد سامانه به‌طور قابل توجهی افزایش بیابد. علاوه بر این، چنین رویکردی می‌تواند به کاهش آسیب‌پذیری‌های امنیتی نیز کمک کند، زیرا از بسیاری از حمله‌های مربوط به خطاهای حافظه جلوگیری خواهد شد.

۲.۱ اهداف پروژه

امروزه برخی از زبان‌های مدرن از نوع خطی^۶ پشتیبانی می‌کنند، ویژگی‌ای که به برنامه‌نویس این امکان را می‌دهد تا مشخص کند یک مقدار دقیقاً یکبار مصرف خواهد شد و پس از استفاده نمی‌تواند دوباره مورد دسترسی قرار بگیرد. این قابلیت باعث می‌شود مدیریت منابعی مانند حافظه، فایل‌ها و اتصالات شبکه به‌شکل ایمن و قابل پیش‌بینی انجام شده و از بروز خطاهایی مانند نشتی حافظه و یا دسترسی نادرست به آن جلوگیری شود. نوع خطی همچنین، به کامپایلر اجازه می‌دهد تا بهینه‌سازی‌های موثرتری انجام دهد، زیرا تضمین می‌کند که این مقادیر غیرقابل اشتراک‌گذاری هستند و نیازی به کپی‌برداری‌های اضافه وجود ندارد.

هدف از این پروژه، طراحی و پیاده‌سازی ابزاری برای زبان برنامه‌نویسی C است تا بتواند با استفاده از اعمال قواعد نوع خطی بر روی متغیرها از مشکلات حافظه جلوگیری نماید. در اینجا منظور از مشکلات حافظه، خطاهای نشتی حافظه، استفاده از حافظه پس از آزادسازی و آزادسازی دوباره حافظه است. از آنجایی که این تحلیل به‌صورت ایستا^۷ صورت می‌گیرد پس دارای انتزاع بدون هزینه بوده و سرباری در زمان اجرا^۸ نخواهد داشت. اهداف این پروژه شامل موارد زیر است:

- ارائه راهکاری دارای انتزاع بدون هزینه به‌منظور بررسی خطاهای مربوط به حافظه در زمان کامپایل^۹ برنامه‌های نوشته شده با استفاده از زبان برنامه‌نویسی C
- بهبود ایمنی و پایداری برنامه‌های نوشته شده توسط زبان برنامه‌نویسی C
- اضافه کردن نوع خطی به زبان برنامه‌نویسی C و گسترش سامانه^{۱۰} نوع این زبان

^۵ garbage collector

^۶ linear type

^۷ static analysis

^۸ runtime

^۹ compile time

^{۱۰} type system

۳.۱ ابزارها و سامانه‌های مشابه

در زبان‌های برنامه‌نویسی، زباله جمع‌کن‌ها^{۱۱} ابزاری هستند که مدیریت حافظه را به‌صورت خودکار انجام می‌دهند. این ابزار در زبان‌های سطح بالا وجود داشته و این امکان را ایجاد می‌کند تا برنامه‌نویس نیازی به تخصیص و آزادسازی حافظه به‌صورت دستی نداشته باشد. زباله جمع‌کن‌ها با شناسایی اشیاء غیرقابل استفاده و آزادسازی حافظه آن‌ها، از بروز خطاهایی مانند نشتی حافظه جلوگیری می‌کنند [۳]. در مقایسه با زباله جمع‌کن‌ها این پروژه دارای برتری‌های زیر است:

- کنترل دقیق‌تر حافظه
 - عدم وجود سربرار اجرا به دلیل تحلیل ایستای برنامه در زمان کامپایل
 - استفاده در کنار ابزارهای توسعه نرم‌افزار
- زبان برنامه‌نویسی سایکلون^{۱۲}، یک نسخه اصلاح شده از زبان برنامه‌نویسی C است که از آن مشتق شده و به مدیریت بهتر حافظه پرداخته است. هدف این زبان رفع مشکلات رایج مربوط به حافظه در زبان برنامه‌نویسی C بدون از بین بردن انعطاف‌پذیری آن است. ویژگی‌های این زبان به‌صورت زیر است:
- ایمنی اشاره‌گرها^{۱۳} : زبان برنامه‌نویسی سایکلون مشکلات مربوط به اشاره‌گرها و مدیریت حافظه را با استفاده از افزودن نوع‌های جدید و گسترش سامانه نوع زبان برنامه‌نویسی C حل کرده است.
 - مدیریت حافظه با تخصیص منطقه‌ای: این ویژگی به برنامه‌نویس این امکان را می‌دهد تا حافظه را در مناطقی تعریف شده تخصیص دهد که با خاتمه کار با متغیر به‌صورت خودکار آزاد می‌شوند.
- در مقایسه با زبان برنامه‌نویسی سایکلون، این پروژه دارای برتری‌های زیر است:
- عدم نیاز به بازنویسی کامل کدهای بجا مانده از قدیم برای استفاده از این ابزار
 - ارائه مکانیزمی ساده‌تر که قابل تطبیق با برنامه‌های فعلی نوشته شده در زبان برنامه‌نویسی C بدون نیازی به بازنویسی پروژه در یک زبان برنامه‌نویسی جدید
 - عدم وجود سربرار اجرا به دلیل تحلیل ایستای برنامه در زمان کامپایل
 - امکان کامپایل کردن برنامه بدون استفاده از این ابزار در صورت نیاز به برنامه‌نویسی غیر ایمن
- ابزار دیگری که به‌منظور بررسی خطاهای مربوط به حافظه از آن استفاده می‌شود، ابزار ولگریند^{۱۴} است. این ابزار یک چارچوب متن‌باز^{۱۵} برای تحلیل پویا در زمان اجرای برنامه بوده و با رهگیری تمامی تخصیص‌ها و آزادسازی‌های حافظه، خطاهایی نظیر دسترسی به بخش‌های نامعتبر حافظه و نشتی‌های حافظه را گزارش می‌دهد.

^{۱۱} garbage collector

^{۱۲} Cyclone

^{۱۳} pointer

^{۱۴} Valgrind

^{۱۵} open source

از آنجا که ولگریند برنامه را در یک محیط شبیه‌سازی اجرا می‌کند، اطلاعات دقیقی از نوع خطا در اختیار توسعه‌دهنده قرار می‌دهد ولی این امر موجب کاهش سرعت اجرای برنامه نسبت به حالت عادی می‌شود. به کمک این ابزار، توسعه‌دهندگان می‌توانند خطاهای پنهان در مدیریت حافظه که معمولاً به سختی قابل شناسایی هستند را به‌طور مؤثر تشخیص داده و برطرف نمایند [۳]. در مقایسه با ابزار ولگریند این پروژه دارای برتری‌های زیر است:

- عدم کاهش سرعت اجرای برنامه به‌دلیل استفاده از تحلیل ایستا
- امکان اثبات‌پذیری صوری
- یکپارچه‌سازی آسان‌تر با زنجیره توسعه نرم‌افزارها
- مقیاس‌پذیری بهتر در پروژه‌های بزرگ
- ارائه بازخورد زودهنگام

ابزار اسموک^{۱۶}، یک تحلیل‌گر ایستا برای زبان برنامه‌نویسی C است که هدف اصلی‌اش پیدا کردن خطاهای مربوط به مدیریت حافظه است. ابزار اسموک سعی می‌کند تا با دنبال کردن مسیرهای مختلف اجرای برنامه و بررسی رفتار توابع تخصیص و آزادسازی حافظه، خطاهای مربوط به حافظه را ردیابی کند. برای مثال اگر در یک تابع حافظه‌ای تخصیص یافته اما در هیچ شاخه‌ای آزادسازی متناظر با آن دیده نشود، اسموک این را به‌عنوان احتمال نشتی حافظه گزارش می‌دهد. اساس کار ابزار اسموک تحلیل تقریبی کد است، به این معنی که این ابزار در اکثر مواقع خطاهای مربوط به حافظه را پیدا کرده ولی همچنان مواردی وجود دارند که خطاهای حافظه داشته باشند ولی ابزار اسموک آن‌ها را معتبر تشخیص می‌دهد [۳].

درمقابل با استفاده از این پروژه، مدیریت حافظه به‌عنوان بخشی از قوانین معنایی خود زبان برنامه‌نویسی C مدل شده و رفتار برنامه تخمین زده نمی‌شود. استفاده از نوع خطی یک چارچوب سخت‌گیرانه ایجاد می‌کند که اگر برنامه از آن عبور کند، با قطعیت می‌توان گفت که مشکلی از جنس نشتی حافظه و دیگر خطاهای مربوط به حافظه در آن وجود ندارد. رویکردهای مبتنی بر نوع خطی مانند آنچه در زبان‌هایی مثل راسط^{۱۷} دیده می‌شود نسبت به تحلیل‌گرهایی مانند اسموک، سطح بالاتری از اطمینان را فراهم می‌کنند.

در ادامه توصیف صوری سامانه نوع پیشنهادی بیان شده و بررسی می‌شود. سپس، براساس توصیف انجام شده، پیاده‌سازی ابزار صورت گرفته و الگوریتم مورد استفاده به‌منظور تشخیص خطاهای حافظه بیان می‌شود. در نهایت ابزار پیاده‌سازی شده مورد سنجش و ارزیابی قرار گرفته و درستی و دقت آن محاسبه می‌شود.

^{۱۶} Smoke

^{۱۷} rust

فصل دوم

مفاهیم پایه

۱.۲ نوع‌ها

در علوم کامپیوتر، نوع^۱ مفهومی است که به یک متغیر نسبت داده شده و مشخص می‌کند که آن متغیر چه نوع مقداری را می‌تواند در خود ذخیره کند و چه عملیات‌هایی بر روی آن مجاز هستند. به عبارت دیگر، نوع تعیین می‌کند که برنامه‌چطور باید با مقادیر مختلف کار کرده و حافظه چگونه باید برای آن‌ها اختصاص بیابد. این مفهوم به برنامه‌نویس‌ها و کامپایلرها کمک می‌کند تا از بروز خطاهای احتمالی در هنگام اجرای برنامه جلوگیری کرده و بسیاری از بررسی‌ها را در زمان کامپایل انجام دهند.

نوع‌های پایه شامل اعداد صحیح، اعداد اعشاری، حروف و نوع بولی^۲ هستند. هریک از این نوع‌ها محدودیت‌ها و قوانین خاص خود را دارند که استفاده صحیح از آن‌ها را ضروری می‌کند. به‌طور مثال، نمی‌توان یک متغیر با نوع بولی و یک متغیر با نوع صحیح را با یکدیگر جمع نمود. با ترکیب نوع‌های پایه، می‌توان نوع‌های پیچیده‌تر مانند ساختارها^۳، آرایه‌ها، رشته‌ها و شی‌ها^۴ را ایجاد نمود. این قابلیت باعث می‌شود تا برنامه‌نویسان بتوانند داده‌های واقعی را مدلسازی کرده و نوع‌های پیچیده‌تری بسازند. زبان‌های مختلف ممکن است یک نوع را با نام‌های مختلفی معرفی کنند، به‌طور مثال، در زبان برنامه‌نویسی جاوا، int نشان‌دهنده اعداد صحیح ۳۲ بیتی و در زبان راسپ ۳۲، نشان‌دهنده این نوع است. همچنین، ممکن است نوع‌هایی با نام یکسان در دو زبان، معنای متفاوتی داشته باشند. به‌طور مثال، int در زبان پایتون نشان‌دهنده عدد صحیح با تعداد بیت متغیر است.

کامپایلرها می‌توانند با در نظر گرفتن نوع داده‌ها، بهینه‌سازی‌های متعددی را بر روی فضای ذخیره‌سازی و الگوریتم‌های مورد استفاده برای انجام عملیات بر روی آن‌ها انجام دهند. به‌طور مثال، در صورتی که نوع یک داده عدد اعشاری باشد، کامپایلر می‌تواند از دستورهای مربوط به اعداد اعشاری در پردازنده برای عملیات‌های انجام شده بر روی این داده استفاده کند. همچنین، در صورتی که نوع یک داده عدد صحیح ۸ بیتی باشد آنگاه کامپایلر می‌تواند الگوریتم‌های جمع و ضرب سریع‌تری برای این داده به کار بگیرد. در سطح پیشرفته‌تر نیز تحلیل نوع داده‌ها می‌تواند به حذف محاسبه‌های غیرضروری و یا پیش‌بینی رفتار برنامه منجر شود. برای نمونه اگر نوع داده تضمین کند که مقدار آن همیشه مثبت است آنگاه کامپایلر می‌تواند از بررسی‌های اضافه برای مثبت بودن این داده صرف نظر کند. در این صورت، نه تنها کارایی برنامه افزایش می‌یابد بلکه مصرف انرژی و حجم کد تولید شده نیز کاهش خواهد یافت.

^۱ type

^۲ boolean

^۳ structure

^۴ objects

پس به‌طور کلی، نوع‌ها ابزار قدرتمندی برای تسهیل بهینه‌سازی‌های سطح پایین و سطح بالا، افزایش ایمنی برنامه و کاهش خطاهای زمان اجرا هستند. علاوه بر این، نوع‌ها به توسعه‌دهندگان کمک می‌کنند تا کد خوانتر و با قابلیت نگهداری بیشتری نوشته و قراردادهایی را مشخص کنند که برنامه مطابق با آن‌ها عمل کند. چنین ویژگی‌ای امکان بهینه‌سازی را افزایش داده و ایمنی و کارایی نرم‌افزارها را افزایش می‌دهد و به طراحی سیستم‌های مطمئن با کارایی بالا کمک می‌کند.

۲.۲ سامانه نوع

یکی از رویکردهای بنیادین در تحلیل برنامه‌ها، بهره‌گیری از سامانه‌های نوع^۵ است. سامانه نوع را می‌توان یک چارچوب صوری تعریف کرد که مجموعه‌ای از قوانین را بر روی عبارت‌ها، مقادیر و متغیرهای یک زبان برنامه‌نویسی تحمیل می‌کند. هدف اصلی چنین سامانه‌ای تضمین ویژگی‌های ایمنی نظیر جلوگیری از عملیات بین نوع‌های نامعتبر، استفاده از مقادیر تعریف نشده و یا فراخوانی توابع با ورودی‌های نادرست است. سامانه نوع به هر عبارت یک نوع نسبت می‌دهد و سپس سازگاری عملیات روی این نوع‌ها را بررسی می‌کند. به‌عنوان مثال، در یک زبان نوع‌دار^۶ عملیات جمع بین یک رشته و یک عدد صحیح می‌تواند توسط سامانه نوع زبان یک خطای ایستا^۷ در نظر گرفته شود. این امر باعث می‌شود تا تعداد زیادی از خطاهای برنامه‌ها در زمان کامپایل برنامه مشخص شوند. در نتیجه، یک سامانه نوع می‌تواند بدون ایجاد سربار در زمان اجرای برنامه از بسیاری از خطاها جلوگیری کند.

سامانه‌های نوع از چندین منظر دسته‌بندی می‌شوند که شامل سامانه نوع ایستا در مقابل پویا، سامانه نوع قوی در مقابل ضعیف و سامانه نوع صریح در مقابل ضمنی است. این دسته‌بندی‌ها به طراحان زبان‌های برنامه‌نویسی کمک می‌کنند تا بر اساس نیازهای حوزه کاربرد، توازن مناسبی میان ایمنی، کارایی و انعطاف‌پذیری زبان برقرار کنند. در یک سامانه نوع ایستا^۸، بررسی نوع‌ها پیش از اجرای برنامه صورت گرفته و هر متغیر یا عبارت، نوع مشخصی به خود می‌گیرد که در سراسر برنامه ثابت است. در یک سامانه نوع پویا^۹، نوع‌ها در زمان اجرای برنامه بررسی می‌شوند و یک متغیر ممکن است در زمان‌های مختلف، مقادیر با نوع‌های متفاوتی داشته باشد. در سامانه‌های نوع قوی^{۱۰}، قوانین نوعی به‌طور سخت‌گیرانه اعمال می‌شوند و تبدیل نوع ضمنی به ندرت مجاز است و در صورت نیاز، برای ترکیب نوع‌های متفاوت برنامه‌نویس باید به‌طور صریح تبدیل را انجام دهد. در سامانه‌های نوع ضعیف^{۱۱}، انعطاف‌پذیری بیشتر بوده و تبدیل‌های ضمنی به‌طور گسترده پشتیبانی می‌شوند. این امر اگرچه ممکن است باعث افزایش سرعت توسعه و راحتی استفاده از

^۵ type system

^۶ typed language

^۷ static error

^۸ static typing

^۹ dynamic typing

^{۱۰} strong typing

^{۱۱} weak typing

زبان شود اما احتمال بروز خطاهای پیش‌بینی نشده را نیز افزایش می‌دهد. در یک سامانه نوع صریح^{۱۲}، برنامه‌نویس باید نوع متغیرها، توابع و مقادیر را به‌طور مستقیم مشخص کند. در یک سامانه نوع ضمنی^{۱۳}، کامپایلر و یا مفسر قادر است تا بر اساس زمینه و استفاده‌های یک مقدار یا متغیر، به‌طور خودکار نوع‌ها را استنتاج^{۱۴} کند.

۱.۲.۲ درستی

درستی^{۱۵} یک سامانه نوع به این معنی است که این سامانه نوع تضمین می‌کند هر برنامه‌ای که توسط قوانین آن پذیرفته می‌شود، در هنگام اجرا با خطاهای مربوط به نوع مواجه نخواهد شد. به بیان دیگر، اگر سامانه نوع برنامه‌ای را معتبر بداند آنگاه آن برنامه حتما معتبر بوده و مثال منفی غلط^{۱۶} وجود نخواهد داشت. درستی سامانه نوع به برنامه‌نویس و طراح زبان این اطمینان را می‌دهد که بسیاری از خطاهای رایج پیش از اجرای برنامه شناسایی می‌شوند. این ویژگی باعث افزایش اعتبار نرم‌افزار نوشته شده با استفاده از زبان شده و احتمال رخداد خطاهای غیرمنتظره در هنگام اجرا را کاهش می‌دهد. همچنین، اگر سامانه نوع درست باشد، می‌توان مطمئن بود که عملیات‌های انجام شده بر روی داده‌ها فقط در چارچوبی صورت می‌گیرند که مجاز است.

از آنجایی که بررسی وجود یک ویژگی غیربديهی برای یک برنامه به‌دلیل مسئله توقف^{۱۷} غیر قابل تصمیم‌گیری است پس تضمین درستی سامانه نوع باعث می‌شود تا بسته به میزان سخت‌گیری سامانه نوع، تعدادی از برنامه‌های درست پذیرفته نشوند با اینکه خطایی ندارند. این موضوع باعث می‌شود تا مثال‌های مثبت غلط^{۱۸} برای سامانه‌های نوع درست وجود داشته باشد. پس در صورتی که سامانه نوعی درست باشد و برنامه‌ای را معتبر بداند آنگاه آن برنامه حتما معتبر بوده و اگر برنامه را نامعتبر بداند، آن برنامه ممکن است نامعتبر نبوده و معتبر باشد. به‌طور کلی، درستی یک سامانه نوع تضمین می‌کند که برنامه‌های معتبر در عمل رفتار پیش‌بینی شده و ایمنی خواهند داشت. با وجود این ممکن است برنامه‌هایی وجود داشته باشند که از نظر سامانه نوع نامعتبر بوده ولی خطایی نداشته باشند. قابل ذکر است که مقاوم بودن سامانه نوع در مقابل نمونه‌های منفی غلط در اکثر موارد مهم‌تر از مقاوم بودن آن نسبت به نمونه‌های مثبت غلط است زیرا هزینه معتبر دانستن تعدادی برنامه نامعتبر، به مراتب بیشتر از نامعتبر دانستن تعدادی برنامه معتبر است.

^{۱۲} implicit typing

^{۱۳} explicit typing

^{۱۴} infer

^{۱۵} soundness

^{۱۶} false negative

^{۱۷} halting problem

^{۱۸} false positive

۲.۲.۲ کامل بودن

کامل بودن^{۱۹} یکی از ویژگی‌های سامانه‌های نوع بوده و به این معنی است که آن سامانه نوع تمامی برنامه‌های معتبر را معتبر دانسته و می‌پذیرد. به بیان دیگر، برنامه‌هایی که توسط سامانه نوع رد می‌شوند حتما دارای خطا بوده و نامعتبر هستند. وجود این ویژگی باعث می‌شود تا نمونه‌های مثبت غلط برای سامانه نوع وجود داشته باشد که در اکثر مواقع خطرناک است. این ویژگی معمولا در مقابل درستی سامانه نوع قرار می‌گیرد. درستی سامانه نوع تضمین می‌کند که برنامه‌های پذیرفته شده در اجرا خطای نوع ندارند ولی کامل بودن سامانه نوع تضمین می‌کند که تمامی برنامه‌هایی که واقعا معتبر هستند، توسط سامانه نوع پذیرفته می‌شوند. با توجه به این دو ویژگی، داشتن هردوی آن‌ها با یکدیگر ایده‌آل است. جدول ۱.۲ مقایسه این دو ویژگی را نشان می‌دهد.

برنامه پذیرفته شده است	برنامه پذیرفته شده است	
برنامه ممکن است معتبر باشد	برنامه حتما معتبر است	سامانه نوع درست است
برنامه حتما نامعتبر است	برنامه ممکن است نامعتبر باشد	سامانه نوع کامل است

جدول ۱.۲: مقایسه درستی با کامل بودن سامانه‌های نوع

کامل نبودن سامانه نوع، یک تصمیم مهم در طراحی زبان است. طراحان زبان معمولا تصمیم می‌گیرند که ایمنی و پیشگیری از خطا مهم‌تر از پذیرش همه برنامه‌های درست است. بنابراین، اکثر زبان‌ها به گونه‌ای طراحی شده‌اند که سامانه نوع آن‌ها درست بوده ولی ناکامل است. با وجود این، تا جای ممکن تلاش می‌شود تا سامانه نوع به کامل بودن نزدیک شده و برنامه‌های درست بیشتری را معتبر بدانند. در غیراین صورت سامانه نوعی که هیچ برنامه‌ای را قبول نمی‌کند نیز درست بوده درحالی که در عمل کاربردی ندارد. اینکه یک سامانه نوع درست است نباید دقت سامانه نوع را در حد غیرقابل استفاده شدن پایین بیاورد و باید به کامل بودن سامانه نوع نیز تا جای ممکن توجه شود. یک سامانه نوع قابل قبول معمولا درست بوده و درجه خوبی از کامل بودن را نیز دارا است.

^{۱۹} completeness

۳.۲ سامانه نوع زیرساختاری

سامانه‌های نوع زیرساختاری^{۲۰} خانواده‌ای از سامانه‌های نوع هستند که قابل قیاس با منطق زیرساختاری بوده و تعدادی از قاعده‌های ساختاری^{۲۱} درونشان وجود ندارد. این سامانه‌های نوع با حذف و یا تغییر تعدادی از قاعده‌های ساختاری امکان مدیریت بهتر و دقیق‌تر منابع را فراهم می‌کنند. سامانه‌های نوع مختلفی در این خانواده به واسطه حذف تعدادی از اصول قاعده‌های ساختاری به وجود آمده‌اند. اصول قاعده‌های ساختاری به‌صورت زیر هستند:

- تضعیف^{۲۲}: این اصل اجازه می‌دهد تا یک فرض یا منبع بدون استفاده باقی مانده و از محاسبه‌ها حذف شود. به‌طور مثال، می‌توان یک متغیر تعریف نموده و از آن استفاده نکرد. این اصل در سامانه نوع زبان‌هایی که معمولاً از آن‌ها استفاده می‌شود، وجود دارد.

- انقباض^{۲۳}: این اصل اجازه می‌دهد تا از یک فرض چندین بار استفاده شود. به‌طور مثال، می‌توان متغیرها و یا منابع را آزادانه جابه‌جا کرده و استفاده نمود. این اصل در سامانه نوع زبان‌هایی که معمولاً از آن‌ها استفاده می‌شود، وجود دارد.

- جابه‌جایی^{۲۴}: این اصل اجازه می‌دهد تا ترتیب فرض‌ها یا منابع در محیط تغییر کند بدون اینکه تاثیری در محاسبه‌ها داشته باشد. این اصل در سامانه نوع زبان‌هایی که معمولاً از آن‌ها استفاده می‌شود، وجود دارد.

با توجه به اینکه از کدام یکی از این اصول استفاده می‌شود، سامانه‌های نوع مختلفی به وجود آمده‌اند که هر یک در حوزه‌ای کاربرد دارند. شکل ۲.۲ انواع سامانه‌های نوع زیرساختاری را نشان می‌دهد.

در سامانه‌های نوع زیرساختاری، تغییر و یا حذف قاعده‌های ساختاری نه‌تنها منجر به ساده‌تر شدن ساختار اثبات‌ها می‌شود، بلکه امکان بیان و تحلیل مفاهیم پیچیده‌ای همچون کنترل منابع، جریان داده‌ها^{۲۵} و بهینه‌سازی محاسبه‌ای را فراهم می‌سازد. به همین دلیل، این خانواده از منطق‌ها به‌طور گسترده در حوزه‌های برنامه‌نویسی تابعی^{۲۶}، طراحی زبان‌های برنامه‌نویسی ایمن از نظر نوع و نیز در سامانه‌های تأیید صوری^{۲۷} استفاده می‌شوند. همچنین، در زبان‌های برنامه‌نویسی هسکل، کلین و راسط نیز به‌منظور بهبود کارایی از آن‌ها استفاده شده است.

^{۲۰} substructural type system

^{۲۱} substructural rule

^{۲۲} weakening

^{۲۳} contraction

^{۲۴} exchange

^{۲۵} data flow

^{۲۶} functional programming

^{۲۷} formal verification

سامانه نوع	جابه‌جایی	تضعیف	انقباض	نحوه استفاده از متغیرها
مرتب	-	-	-	دقیقا یکبار به ترتیب معرفی
خطی	مجاز	-	-	دقیقا یکبار
آفینی	مجاز	مجاز	-	حداکثر یکبار
وابسته	مجاز	-	مجاز	حداقل یکبار
معمولی	مجاز	مجاز	مجاز	دلخواه

جدول ۲.۲: انواع سامانه‌های نوع زیرساختاری

۴.۲ سامانه نوع خطی

سامانه نوع خطی جزو خانواده سامانه‌های نوع زیرساختاری بوده که در آن هر متغیر دقیقا یکبار باید استفاده شود [۶]. این ویژگی، از رها شدن متغیرها جلوگیری می‌کند. سامانه‌های نوع خطی محدودیت‌های زیر از اصول قاعده‌های ساختاری را اعمال می‌کنند:

- ممنوعیت تضعیف: نمی‌توان منابع یا متغیرها را کنار گذاشته و از محاسبه‌ها حذف نمود، بلکه حتما باید استفاده شوند.
- ممنوعیت انقباض: نمی‌توان منابع یا متغیرها را بیش از یکبار استفاده نمود.
- جابه‌جایی: ترتیب استفاده از منابع اهمیت نداشته و می‌توان با هر ترتیبی از آن‌ها استفاده نمود.

این سامانه نوع بر پایه منطق خطی^{۲۸} بوده و از مدیریت منابع سختگیرانه‌ای با تضمین مصرف متغیر در عملیات انتقال^{۲۹} بهره‌مند است. با استفاده از اشاره‌گرهای خطی می‌توان از خطاهای زیر جلوگیری نمود:

- نشتی حافظه^{۳۰}: با تضمین استفاده یکباره از اشاره‌گر، اشاره‌گر حتما مصرف شده و مقدار آن منتقل می‌شود. در نتیجه، اشاره‌گری باقی نمی‌ماند که رها شده باشد.
- استفاده از حافظه پس از آزادسازی^{۳۱}: از آنجایی که پس از مصرف و آزادسازی اشاره‌گر مقدار آن نامعتبر می‌شود پس اجازه استفاده از حافظه پس از آزادسازی آن داده نخواهد شد.

^{۲۸}linear logic

^{۲۹}move

^{۳۰}memory leakage

^{۳۱}use after free

^{۳۲}double free

- آزادسازی دوباره حافظه^{۳۲} : در صورتی که فرض شود آزادسازی حافظه عملیاتی است که نیاز دارد تا اشاره گر را مصرف کند، با مصرف یکباره اشاره گر و آزادسازی حافظه نمی توان دوباره آن را مصرف نموده و حافظه مورد نظر را دوباره آزاد نمود.

استفاده از این قوانین سختگیرانه در عمل به خصوص در زبان برنامه نویسی C که از روی عمد سامانه نوع آن سست^{۳۳} طراحی شده است محدودکننده بوده و دقت سامانه نوع را پایین می آورد. در نتیجه، به منظور کاهش این محدودیت، عملیات جدیدی به نام قرض گرفتن^{۳۴} نوع خطی تعریف شده است که با استفاده از آن می توان مقدار اشاره گر را بدون مصرف شدن به عنوان نوع غیرخطی قرض گرفت. عملیات قرض گرفتن نوع خطی باعث می شود تا بدون نیاز به مصرف اشاره گر، آن را به توابع دیگر به عنوان نوع غیرخطی ورودی داده و از آن استفاده های دیگری نمود.

۵.۲ ایمنی زبان

تمامی زبان های سطح بالا انتزاع هایی را بر روی ماشین تعریف می کنند. ایمنی زبان به قابلیت زبان برای محافظت از این انتزاع اشاره داشته و زبان های ایمن زبان هایی هستند که از انتزاع خودشان محافظت می کنند. همچنین، ایمنی به معنای تمامیت و برقراری این انتزاع ها و دیگر انتزاع های سطح بالای تعریف شده توسط برنامه نویس با استفاده از قابلیت های زبان نیز است. به طور مثال، یک زبان ممکن است آرایه ها به همراه عملیات های دسترسی و بروزرسانی را به عنوان انتزاعی از حافظه ارائه دهد. برنامه نویسی که از این زبان استفاده می کند انتظار دارد که حافظه مربوط به این آرایه فقط توسط عملیات های مشخص شده بر روی آن تغییر کرده و با نوشتن پس از انتهای ساختمان داده دیگری، تغییری نکند [۴].

زبان های ایمن لزوما دارای بررسی ایستای نوع ها^{۳۵} نبوده و ایمنی زبان می تواند با استفاده از بررسی ایستای نوع ها و یا بررسی های مربوط به زمان اجرا به دست بیاید. همچنین، برخی زبان های غیرایمن نیز ممکن است بررسی ایستای نوع ها را به منظور جلوگیری از اشتباهات رایج به کار بگیرند ولی همچنان ایمن نباشند [۵]. جدول ۳.۲ مثال هایی از زبان های ایمن و غیرایمن با توجه به روش های بررسی نوع ها می دهد.

بررسی ایستا	بررسی پویا	
ایمن	ML, Haskell, Java, Rust	Lisp, Scheme, Perl
ناایمن	C, C++	JavaScript

جدول ۳.۲: مثال هایی از زبان های ایمن و غیرایمن با توجه به روش های بررسی نوع ها

^{۳۳} loose

^{۳۴} borrow

^{۳۵} static type checking

ایمنی زبان باعث قابل پیش‌بینی‌تر و پایدارتر شدن برنامه‌های نوشته شده با استفاده از آن شده و امکان تشخیص خطاهای رایج را قبل از اجرای برنامه ایجاد می‌کند. همچنین، سیستم در برابر حمله‌ها و بد به کار بردن امنیتی مقاوم شده و ضریب اطمینان آن افزایش می‌یابد. به‌طور کلی نیز ایمنی زبان یک ویژگی اساسی برای برنامه‌های قابل اعتماد و سیستم‌های امن است.

پایه‌ای‌ترین ویژگی یک سامانه نوع، ایمنی یا درستی آن سامانه نوع است. یک سامانه نوع در صورتی ایمن است که دو شرط زیر در آن برقرار باشند:

- پیشرفت^{۳۶}: یک اصطلاح خوش نوع در این سامانه نوع گیر نمی‌کند. یعنی یک اصطلاح یا یک مقدار است و یا می‌تواند توسط قاعده‌های ارزیابی در یک قدم پیشرفت داشته باشد [۴].
- حفظ^{۳۷}: اگر یک اصطلاح خوش نوع در یک قدم ارزیابی شود، نتیجه نیز خوش نوع خواهد بود [۴].

۶.۲ خطاهای حافظه

خطای حافظه زمانی رخ می‌دهد که یک برنامه به‌صورت نادرست و غیرمجاز به حافظه دسترسی پیدا کند. مثلاً زمانی که حافظه آزاد شده باشد و یا دسترسی از محدوده آرایه فراتر برود. این خطاها معمولاً باعث رفتار غیرقابل پیش‌بینی در برنامه، از کار افتادن برنامه و یا گیر کردن سیستم می‌شوند. همچنین، ممکن است در برخی موارد هشدارهای ایجاد نشده اما داده‌ها به اشتباه پردازش شوند و رفتار منطقی برنامه دچار تغییر شود. تعدادی از انواع حمله‌هایی که مربوط به خطاهای حافظه هستند در ادامه آورده شده است:

- سرریز بافر^{۳۸}: با نوشتن فراتر از فضای اختصاص داده شده به یک بافر می‌توان به حافظه مربوط به دیگر متغیرها دسترسی پیدا کرده و در مقدار آن‌ها تغییر ایجاد نمود.
- خرد کردن پشته^{۳۹}: با تغییر آدرس بازگشت ذخیره شده در پشته می‌توان به توابع دیگر برنامه و یا بخش‌های غیرمجاز آن دسترسی پیدا کرد.
- برنامه‌نویسی بازگشت‌گرا^{۴۰}: در این حمله، به‌صورت زنجیره‌ای آدرس بازگشت ذخیره شده در پشته تغییر پیدا کرده و با اتصال بخش‌ها و یا حتی دستورهای مختلف برنامه، عملیات دلخواه انجام می‌شود.
- سرریز اعداد صحیح^{۴۱}: با سرریز متغیری که اعداد صحیح در خود نگه می‌دارد می‌توان مقدار غیرمجاز در آن قرار داده تا در ادامه برنامه از آن استفاده شود.

^{۳۶} progress

^{۳۷} preservation

^{۳۸} buffer overflow

^{۳۹} stack smashing

^{۴۰} return-oriented programming

^{۴۱} integer overflow

خطاهای حافظه یکی از مهم‌ترین دلایل آسیب‌پذیری امنیتی در برنامه‌ها هستند. افراد سودجو با بهره‌گیری از این خطاها می‌توانند برنامه را فریب داده و یا به‌طور غیرمجاز به منابع سیستم دسترسی پیدا کنند. از طرفی، وقوع خطاهای حافظه باعث افزایش هزینه توسعه نیز می‌شود زیرا پیدا کردن این خطاها دشوار و هزینه‌بر است. در محیط‌های صنعتی و مالی نیز چنین خطاهایی می‌توانند باعث خسارت مستقیم و توقف سرویس‌ها شوند. چنین خطاهایی ممکن است به‌صورت محسوس بروز نکنند و با گذر زمان باعث خرابی‌های پیچیده شوند. تحقیق‌ها و آمارها نشان می‌دهند که حدود ۷۰ تا ۸۰ درصد آسیب‌پذیری‌های امنیتی در جهان مربوط به خطاهای حافظه بوده و ۷۵ درصد حمله‌های روز صفر^{۴۲} از این خطاها نشأت می‌گیرند.

۱.۶.۲ نشتی حافظه

نشتی حافظه زمانی رخ می‌دهد که برنامه حافظه‌ای در طول اجرا تخصیص داده ولی در ادامه آن را آزاد نمی‌کند حتی اگر دیگر نیازی به آن نداشته باشد. این بخش از حافظه که آزاد نشده است دیگر قابل استفاده نبوده و نمی‌تواند دوباره تخصیص پیدا کند. به همین دلیل، با ادامه این روند به تدریج ممکن است حافظه پر شده و برنامه متوقف شود. تعدادی از اثرات نشتی حافظه در ادامه آورده شده است:

- کاهش عملکرد برنامه: حافظه در دسترس کاهش کرده و ممکن است برنامه کند شود.
 - توقف برنامه: اگر نشتی حافظه زیاد باشد به تدریج حافظه پر شده و برنامه متوقف خواهد شد.
 - افزایش مصرف منابع: با افزایش مصرف منابع، فشار وارده به سیستم عامل و حافظه افزایش می‌یابد.
- نشتی حافظه ممکن است به دلایل متعددی ایجاد شود. در ادامه تعدادی از دلایل ایجاد نشتی حافظه آورده شده است:
- عدم آزادسازی حافظه اختصاص یافته
 - از دست رفتن اشاره‌گر حافظه و عدم دسترسی به آن
 - ذخیره‌سازی‌های غیرضروری بدون آزادسازی

نشتی حافظه معمولاً در زبان‌های برنامه‌نویسی سطح پایین مانند C رخ می‌دهد زیرا در این زبان‌ها مدیریت حافظه به‌طور خودکار صورت نگرفته و مسئولیت کامل تخصیص و آزادسازی حافظه به عهده برنامه‌نویس است. در مقابل زبان‌های سطح بالاتر مانند جاوا و پایتون مجهز به سازوکارهای مدیریت حافظه خودکار مثل زباله جمع‌کن‌ها هستند که به‌صورت دوره‌ای، حافظه‌های بدون استفاده را شناسایی و آزاد می‌کند. در نتیجه، هرچند نبود مدیریت خودکار حافظه قدرت و انعطاف‌پذیری زیادی به برنامه‌نویس می‌دهد ولی خطر بروز خطاهای جدی مانند نشتی حافظه را نیز افزایش خواهد داد.

^{۴۲} zero-day exploit

۲.۶.۲ استفاده از حافظه پس از آزادسازی

خطای استفاده از حافظه پس از آزادسازی زمانی رخ می‌دهد که حافظه‌ای تخصیص داده شده و آزاد شده باشد و سپس دوباره از آن حافظه استفاده شود. به عبارت دیگر، اشاره‌گر حافظه آزاد شده همچنان در دسترس بوده و از آن برای خواندن یا نوشتن بر روی حافظه استفاده می‌شود. معمولاً در زبان‌های برنامه‌نویسی سطح پایین، یک حافظه نهان وجود دارد که قطعه‌های حافظه آزاد شده را نگه داشته و در تخصیص‌های بعدی از آن‌ها استفاده می‌کند. وجود چنین سازوکاری باعث تخصیص سریع‌تر حافظه شده و کارایی برنامه را افزایش می‌دهد ولی زمینه‌ساز بد به کار بردن بعضی از خطاهای حافظه نیز است. شکل ۱.۳ نمونه‌ای از بد به کار بردن دسترسی به حافظه پس از آزادسازی را نشان می‌دهد.

```
main.c

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr_1 = malloc(sizeof(int) * 8);
    free(ptr_1);

    int *ptr_2 = malloc(sizeof(int) * 8);
    for (int i = 0; i < 8; i++) {
        ptr_2[i] = i;
    }

    for (int i = 0; i < 8; i++) {
        printf("%d ", ptr_1[i]);
    }
    printf("\n");

    return 0;
}
```

شکل ۱.۳: بد به کار بردن دسترسی به حافظه پس از آزادسازی

با توجه به شکل ۱.۳، ابتدا حافظه‌ای به اندازه ۳۲ بایت تخصیص داده و آزاد شده و سپس در ادامه دوباره حافظه‌ای با همین اندازه تخصیص داده شده است. در اینجا به دلیل وجود سازوکار حافظه نهان برای تخصیص سریع‌تر حافظه، حافظه قبلی دوباره تخصیص داده می‌شود. در ادامه، داده جدیدی در حافظه ذخیره شده و این داده توسط اشاره‌گر قبلی به دلیل وجود خطای دسترسی به حافظه پس از آزادسازی خروجی داده شده است. با توجه به این مثال، استفاده از حافظه پس از آزادسازی بسیار خطرناک بوده و زمینه‌ساز بسیاری از مشکلات امنیتی است.

۳.۶.۲ آزادسازی دوباره حافظه

این خطای حافظه زمانی رخ می‌دهد که حافظه‌ای تخصیص داده شده و آزاد شده باشد و سپس در ادامه دوباره آزاد شود. در این صورت، به دلیل وجود سازوکار حافظه نهان به منظور تخصیص سریع‌تر حافظه، این قطعه از حافظه دوبار در این حافظه نهان قرار گرفته و دو تخصیص بعدی با سایز یکسان، به همین قطعه از حافظه اشاره خواهند نمود. شکل ۲.۳ نمونه‌ای از بد به کار بردن آزادسازی دوباره حافظه را نشان می‌دهد.

```
main.c

#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr_1 = malloc(sizeof(int) * 8);
    free(ptr_1);
    free(ptr_1);

    int *ptr_2 = malloc(sizeof(int) * 8);
    int *ptr_3 = malloc(sizeof(int) * 8);

    printf("ptr_1: %d\n", ptr_1);
    printf("ptr_2: %d\n", ptr_2);
    printf("ptr_3: %d\n", ptr_3);

    return 0;
}
```

شکل ۲.۳: بد به کار بردن آزادسازی دوباره حافظه

با توجه به شکل ۲.۳، ابتدا حافظه‌ای به اندازه ۳۲ بایت تخصیص داده شده و سپس دوبار آزاد شده است. در اینجا چون این قطعه از حافظه دوبار آزاد شده است پس دو بار در حافظه نهان برای دسترسی سریع‌تر قرار می‌گیرد. در ادامه دوبار حافظه با سایز یکسان تخصیص داده شده و سپس آدرس سه اشاره‌گر خروجی داده شده است که همگی یکسان هستند.

۷.۲ تحلیل برنامه

تحلیل برنامه با هدف بررسی خواص و رفتارهای برنامه‌ها انجام می‌شود. تحلیل برنامه می‌تواند روی برنامه‌های خاص یک حوزه و یا برنامه‌های عمومی انجام شود. تحلیل‌های حوزه‌ای^{۴۳} بر روی خانواده‌ای از برنامه‌ها با ویژگی‌های مشخص تمرکز داشته ولی تحلیل‌های عمومی بدون تمرکز بر یک حوزه خاص، معمولاً در محیط‌های برنامه‌نویسی یا کامپایلرها به‌منظور بهینه‌سازی و یا کشف خطا استفاده می‌شوند. تحلیل برنامه می‌تواند بر روی کد منبع برنامه^{۴۴} و یا بر روی مدل برنامه نیز صورت بگیرد. تحلیل کد برنامه معمولاً شامل تجزیه نحوی مشابه کامپایلر و بررسی مستقیم کد است. خواصی که در تحلیل‌ها مورد بررسی قرار می‌گیرند، می‌توانند شامل موارد زیر باشند [۵]:

- ایمنی: این خواص بیان می‌کنند که برنامه هرگز رفتار نامطلوب در زمان محدود از خود نشان ندهد. جلوگیری از سرریز بافر، جلوگیری از تقسیم بر صفر و اجتناب از بن‌بست‌ها^{۴۵} مثال‌هایی از این خواص هستند. تحلیل‌های ایمنی معمولاً روی اجراهای محدود و کوتاه‌مدت برنامه تمرکز داشته و هدف آن‌ها اثبات نبود رفتارهای بد در برنامه است.
- زنده‌مانی^{۴۶}: این خواص بیان می‌کنند که برنامه در طول زمان نامحدود، رفتارهای مشخصی را از خود نشان می‌دهد یا از آن‌ها اجتناب می‌کند. عدم توقف ناگهانی و جلوگیری از قحطی منابع^{۴۷} مثال‌هایی از این خواص هستند. تحلیل‌های زنده‌مانی برنامه نیازمند بررسی اجراهای طولانی یا بینهایت برنامه هستند و اغلب پیچیده‌اند.
- جریان اطلاعات^{۴۸}: این خواص به بررسی وابستگی یا عدم وابستگی بین داده‌ها و رفتارهای برنامه می‌پردازند. عدم امکان استخراج رمزعبور یک کاربر توسط دیگر کاربرها از داده‌های قابل استخراج، مثالی از این خواص است. برخلاف دو مورد قبلی، تحلیل‌های جریان اطلاعات معمولاً نیازمند بررسی چند اجرای برنامه به‌طور همزمان هستند و به کلاس بزرگتری از خواص به‌نام بیش ویژگی‌ها^{۴۹} تعلق دارند.

^{۴۳} domain specific

^{۴۴} source code

^{۴۵} deadlock

^{۴۶} liveness

^{۴۷} starvation

^{۴۸} information flow

^{۴۹} hyperproperties

۱.۷.۲ تحلیل ایستا

تحلیل ایستا به بررسی کد منبع و یا کد کامپایل شده برنامه قبل از اجرای آن پرداخته و مستقل از هر اجرای واقعی برنامه انجام می‌شود. این نوع تحلیل شامل بررسی ساختارهای نحوی، جریان اطلاعات، وابستگی‌ها و نوع داده‌ها است. به عنوان مثال، کامپایلرها معمولاً مرحله‌ای برای بررسی نوع داده‌ها دارند که تضمین می‌کند برخی خطاها مانند استفاده نادرست از نوع‌ها و یا مقادیر در زمان اجرا رخ نمی‌دهند. در واقع تحلیل ایستا تلاش می‌کند تا قبل از اجرای واقعی برنامه، خطاها و ناسازگاری‌ها را شناسایی کرده تا از بروز رفتارهای ناخواسته جلوگیری شود.

در تحلیل ایستا پس از انجام تحلیل، اجرای برنامه بدون کاهش کارایی ادامه پیدا می‌کند. همچنین، با استفاده از این نوع تحلیل می‌توان خطاها را قبل از انتشار نرم‌افزار برطرف نمود. علاوه بر این، تحلیل‌های ایستا می‌توانند در ابزارهای توسعه و کامپایلرها به طور خودکار اجرا شده و اطلاعاتی برای بهینه‌سازی و ارتقای امنیت برنامه فراهم کنند. تحلیل ایستا با اینکه مزایای زیادی دارد ولی ممکن است دقت کمتری از تحلیل پویا داشته باشد و قادر به بررسی برخی خواص برنامه‌ها نباشد. همچنین، این نوع تحلیل ممکن است اختراهای مثبت غلط داشته و دارای پیچیدگی‌های بسیاری باشد.

۲.۷.۲ تحلیل پویا

تحلیل پویا فرآیندی است که در زمان اجرای برنامه انجام شده و رفتار واقعی برنامه را در هنگام اجرا مورد بررسی قرار می‌دهد. برخلاف تحلیل ایستا که مستقل از اجرای برنامه است، تحلیل پویا وابسته به ورودی‌ها، مسیرهای اجرایی و شرایط زمان اجرا است. این روش امکان شناسایی خطاهایی که فقط در شرایط اجرا ظاهر می‌شوند را فراهم کرده و برای بررسی رفتارهای برنامه در محیط عملی مناسب است.

تحلیل پویا به توسعه‌دهنده اجازه می‌دهد تا خطاها و ناسازگاری‌های واقعی برنامه را مشاهده و ارزیابی کند. این نوع تحلیل معمولاً طراحی ساده‌تری نسبت به تحلیل ایستا داشته و می‌تواند به سرعت اطلاعات ارزشمندی درباره وضعیت برنامه در زمان اجرا ارائه دهد. علاوه بر این، تحلیل پویا برای بررسی مواردی که به شدت وابسته به شرایط زمان اجرا یا ورودی‌های خاص هستند، بسیار مفید است. در تحلیل پویا، اجرای بررسی‌های هم‌زمان با اجرای برنامه باعث افزایش بار پردازشی و کاهش کارایی می‌شود. علاوه بر این، تحلیل پویا نمی‌تواند همه خواص برنامه به‌ویژه خواص مرتبط با اجرای بی‌نهایت یا رفتارهای بلندمدت را پوشش دهد. در این نوع تحلیل، خطاهایی که در مسیرها یا ورودی‌های آزمایش نشده رخ می‌دهند ممکن است شناسایی نشده و دقت تحلیل، بسته به تعداد و تنوع برنامه‌های اجرا شده، متفاوت باشد.

فصل سوم

توصیف صوری سامانهٔ نوع پیشنهادی

۱.۳ واری نوع

واری نوع^۱ به معنای بررسی سازگاری یک برنامه با قواعد سامانه نوع زبان است. در این فرآیند، معتبر بودن و هماهنگی برنامه با قوانین زبان بررسی شده و به طور کلی خوش تایپ^۲ بودن مورد توجه قرار می گیرد. در نهایت اگر برنامه دارای خطاهای نوعی باشد، سیستم باید خطاها را گزارش نماید. تفاوت مهمی که بین واری نوع و استنتاج نوع^۳ وجود دارد، صریح یا ضمنی بودن نوعها است. در واری نوع، نوعها به طور صریح داده می شوند ولی در استنتاج نوع، سامانه نوع باید نوعها را از روی ساختار برنامه و استفاده های متغیرها، استنتاج کند. در چارچوب نظری، واری نوع معمولاً در قالب قواعد استنتاجی^۴ بیان می شود.

از آنجایی که توصیف صوری زبان برنامه نویسی C پیچیده بوده و بررسی قوانین مربوط به نوع خطی را سخت می کند، یک زیرمجموعه از این زبان دارای ویژگی های اصلی آن انتخاب شده و توصیف سامانه نوع بر روی این زیرمجموعه صورت گرفته است. این زیرمجموعه باید دارای ویژگی های اصلی زبان بوده و تورینگ کامل^۵ باشد. همچنین، نیاز است تا ویژگی هایی از زبان که ممکن است در بحث نشستی حافظه دخالت داشته باشند لحاظ شده و مورد بررسی قرار بگیرند. به طور کلی، قواعد زیر در زیرمجموعه انتخاب شده قرار دارند:

- تعریف و استفاده از توابع به صورت محدود و فقط با یک ورودی
- شرطها و حلقه ها به عنوان قواعد کنترل جریان برنامه
- تعریف متغیر و تخصیص مقدار به آن
- عملیات های دوگانی و یگانی پایه
- نوع های پایه زبان شامل اعداد صحیح، نوع بولی، اشاره گر ها و توابع
- نوع خطی به عنوان یکی از نوع های زبان
- مقادارهای صحیح، درست، غلط و واحد

در ادامه با استفاده از قواعد استنتاجی، توصیف صوری این زیرمجموعه از زبان برنامه نویسی C با تمرکز بر قوانین نوع خطی بیان شده و تحلیل می شود. توصیف این زیرمجموعه، با توصیف نحو آن شروع شده و در ادامه قاعده های ارزیابی و نوع دهی مورد نیز بیان می شوند. این زیرمجموعه از زبان C، به عنوان Linear-C معرفی شده و از این نام برای راحتی استفاده شده است.

^۱ type checking

^۲ well-typed

^۳ type inference

^۴ inference rule

^۵ turing complete

۲.۳ نحو

در ابتدا نیاز است تا نحو زبان Linear-C توصیف شده تا بتوان قاعده‌های ارزیابی و قاعده‌های نوع‌دهی را بر روی آن تعریف نمود. به‌طور کلی این زیرمجموعه شامل سه جزء اصلی شامل عبارت‌ها، حکم‌ها و برنامه است، همچنین، نوع‌ها، مقادارها و عملیات‌های دوگانی و یگانی نیز در آن قرار دارند. نوع، مجموعه‌ای از مقادیر با ویژگی‌های مشابه را مشخص می‌کند و نشان می‌دهد که چه عملیات‌هایی بر روی آن‌ها مجاز است. مقدار نیز داده‌ای است که یک عبارت به خود می‌گیرد یا یک متغیر ذخیره می‌کند. شکل ۱.۷ توصیف نحو نوع‌ها و مقادارها را نشان می‌دهد. نوع‌های تعریف شده در زبان Linear-C شامل نوع واحد، نوع اعداد صحیح، نوع بولی، اشاره‌گرها، نوع خطی و توابع بوده و مقادارها نیز شامل مقدار واحد، مقادارهای صحیح، مقدار درست و مقدار غلط هستند.

<i>Type's syntactic forms</i>		<i>Value's syntactic forms</i>	
Type::=	types:	value::=	values:
Unit	unit type	true	constant true
Int	int type	false	constant false
Bool	bool type	unit	constant unit
Type*	pointer type	int	constant integers
Linear Type	linear type		
Type → Type	function type		

شکل ۱.۳: توصیف نحو نوع‌ها و مقادارها

عبارت‌ها در زبان Linear-C ترکیبی از مقادارها، متغیرها، عملگرها و توابع هستند که وقتی اجرا می‌شوند، یک مقدار تولید می‌کنند. عبارت‌ها شامل عملیات‌های دوگانی و یگانی بر روی دیگر عبارت‌ها، متغیرها، مقادارها و فراخوانی توابع هستند. با ترکیب این عبارت‌های ساده می‌توان عبارت‌های پیچیده‌تر به‌منظور انجام محاسبه‌های مختلف در این زبان ایجاد نمود [۱]، [۲]. عملیات‌های دوگانی شامل جمع، عطف منطقی، فصل منطقی، برابری، بزرگتر و کوچکتر بوده و عملیات‌های یگانی نیز شامل منفی، نقیض، عملگر آدرس و عملگر در آدرس^۶ هستند. شکل ۲.۳ توصیف نحوی عبارت‌ها و عملیات‌ها را نشان می‌دهد.

^۶dereference

<i>Expression's syntactic forms</i>		<i>Binary operation's syntactic forms</i>	
expr::=	expressions:	bin_op::=	binary operations:
(expr)	parenthesized	+	addition
expr bin_op expr	binary	&&	and
uni_op expr	unary		or
ident	identifier	==	equal
value	value	<	less than
ident(expr)	function call	>	greater than
		<i>Unary operation's syntactic forms</i>	
		un_op::=	unary operations:
		-	negative
		!	not
		&	address of
		*	dereference

شکل ۲.۳: توصیف نحو عبارتها و عملیاتها

حکمها در زبان Linear-C دستورهای اجرایی ای هستند که می‌توانند ساده و یا مرکب بوده و جریان اجرای برنامه را کنترل نمایند. حکمها معمولاً با نقطه ویرگول^y به پایان رسیده و برنامه آنها را اجرا می‌کند. حکمها شامل شرطها و حلقهها به منظور کنترل جریان برنامه، تعریف متغیر و مقداردهی به آن، تغییر دامنه[^] و ساخت بلوک و بازگرداندن یک مقدار از تابع هستند [۱]، [۲]. شکل ۳.۳ توصیف نحو حکمها را نشان می‌دهد. برنامه نیز دنباله‌ای از توابع است که می‌تواند خالی هم باشد. توابع در زبان Linear-C تنها یک ورودی دریافت می‌کنند و بدنه‌شان یک حکم است زیرا بدنه یک تابع مجموعه‌ای از دستورهای اجرایی است. شکل ۴.۳ توصیف نحو برنامه را نشان می‌دهد.

^y semicolon[^] scope

Statement's syntactic forms

stmt::=	statements:
{ stmt }	compound statement
stmt stmt	sequence
if (expr) stmt else stmt	if statement
while (expr) stmt	while statement
expr;	expression statement
Type ident;	declaration
ident = expr;	assignment
return expr;	return statement
;	semicolon statement

شکل ۳.۳: توصیف نحو حکم‌ها

Program's syntactic forms

program::=	program:
Type ident(Type ident) stmt program	function declaration
ϵ	empty-program

شکل ۴.۳: توصیف نحو برنامه

با درکنار هم قرار دادن این اجزاء، می‌توان یک برنامه کامل در زبان Linear-C نوشته و آن را اجرا نمود. همانطور که نشان داده شد، در این زبان امکان تعریف متغیرها به صورت خطی وجود داشته و از نوع خطی پشتیبانی می‌شود. همچنین، حداقل نحو مورد نیاز از زبان برنامه‌نویسی C برای تورینگ کامل بودن در این زبان لحاظ شده و اشاره‌گرها نیز در این زبان وجود دارند. در ادامه قاعده‌های ارزیابی برای این زبان بیان شده و بررسی می‌شوند.

^۹evaluation rules

۳.۳ قاعده‌های ارزیابی

در حوزه زبان‌های برنامه‌نویسی، قاعده‌های ارزیابی^۹ مجموعه قوانین صریح و دقیقی هستند که مشخص می‌کنند چگونه عبارت‌ها و دستورهای برنامه در زمان اجرا رفتار کرده و چه مقداری می‌گیرند. به عبارت دیگر، این قواعد مشخص می‌کنند که وقتی یک برنامه نوشته می‌شود، کامپیوتر یا مفسر^{۱۰} چگونه آن برنامه را گام به گام اجرا می‌کند. قاعده‌های ارزیابی بخش مهمی از معنای رسمی^{۱۱} زبان را تشکیل می‌دهند. معنای رسمی به برنامه‌نویسان و طراحان زبان اجازه می‌دهد تا رفتار برنامه را توصیف کرده و درستی برنامه‌ها را اثبات کنند. به‌طور خلاصه قاعده‌های ارزیابی در زبان‌های برنامه‌نویسی موارد زیر را مشخص می‌کنند:

- هر عبارت یا دستور چگونه ساده می‌شود.
- ترتیب و روش اجرای دستورها چگونه است.
- پایه‌ای برای تحلیل و اثبات خواص برنامه فراهم می‌کنند.
- نتیجه هر دستور در نهایت چه چیزی است.

به‌منظور توصیف قاعده‌های ارزیابی برای زبان Linear-C، ابتدا نیاز است تا دو مفهوم محیط^{۱۲} و مجموعه توابع تعریف شوند. محیط مجموعه‌ای از زوج‌های شامل شناسه و مقدار است که نشان می‌دهد در هر متغیر در وضعیت فعلی، چه مقداری ذخیره شده است. مجموعه توابع نیز ورودی و بدنه توابعی که تعریف شده‌اند را در خود ذخیره می‌کند. تعریف این دو مفهوم به‌منظور توصیف قاعده‌های ارزیابی ضروری بوده زیرا نیاز است تا در زمان فراخوانی توابع، به آن‌ها دسترسی پیدا کرده و بدنه‌شان با توجه به ورودی ارزیابی شود. با تعریف مفاهیم محیط و مجموعه توابع، می‌توان قاعده‌های ارزیابی عبارت‌ها، حکم‌ها و برنامه را به‌صورت رسمی و دقیق توصیف نمود. شکل ۵.۳ توصیف محیط و مجموعه توابع را نشان می‌دهد.

<i>Env's definition</i>		<i>Fn's definition</i>	
Env::=	enviroment:	Fn::=	functions:
Env, ident:value	value binding	Fn, ident:(ident, stmt)	function binding
ε	empty enviroment	ε	empty functions

شکل ۵.۳: توصیف محیط و مجموعه توابع

^{۱۰} interpreter

^{۱۱} formal semantics

^{۱۲} enviroment

با توجه به شکل ۶.۳ که قاعده‌های ارزیابی عبارت‌ها را نشان می‌دهد، ارزیابی انواع عبارت‌های توصیف شده در بخش قبلی، به صورت زیر است:

- در صورتی که عبارتی دارای پرائتز باید، آن عبارت در یک قدم ارزیابی شده و پرائتز باقی می‌ماند.
 - عبارت‌هایی که در یک عملیات دوگانی وجود دارند، در یک قدم ارزیابی می‌شوند.
 - عبارتی که یک عملیات یگانی بر روی آن انجام می‌شود، در یک قدم باید ارزیابی شود.
 - ورودی یک فراخوانی تابع، باید در یک قدم ارزیابی شود.
 - در صورتی که یک عبارت به یک شناسه ساده شود، اگر شناسه در محیط وجود داشته باشد، مقداری که برایش در محیط ذخیره شده است، بازگردانده می‌شود.
 - در صورتی که قاعده‌های قبلی قابل اعمال نباشند، یعنی عبارت درون پرائتز، به یک مقدار ساده شده است. پس پرائتز برداشته شده و مقدار به دست آمده از آن خارج می‌شود.
 - در صورتی که قاعده‌های قبلی قابل اعمال نباشند، یعنی عبارت موجود در یک عملیات یگانی به یک مقدار ساده شده است. پس عملیات یگانی بر روی آن اجرا شده و یک مقدار جدید به دست می‌آید.
 - در صورتی که قاعده‌های قبلی قابل اعمال نباشند، یعنی عبارت‌های موجود در یک عملیات دوگانی به مقدار ساده شده‌اند. پس عملیات دوگانی بر روی آن‌ها اجرا شده و یک مقدار جدید به دست می‌آید.
 - در یک فراخوانی تابع بعد از ساده شدن ورودی به یک مقدار، نام ورودی تابع و خود بدنه تابع از مجموعه توابع با توجه به شناسه تابع استخراج شده و این بدنه در یک محیط جدید شامل ورودی تابع، ارزیابی می‌شود.
- توابع `binary_eval` و `unary_eval` که در این بخش از آن‌ها استفاده شده است، در حقیقت عملیات ورودی را بر روی مقدارهای داده شده اعمال کرده و در نهایت یک مقدار بازمی‌گردانند. به طور مثال، با اعمال عملیات جمع بر روی دو عدد صحیح، آن دو عدد جمع شده و نتیجه بازگردانده می‌شود. قابل توجه است که ترتیب اعمال این قاعده‌ها دارای اهمیت بوده و باید به این ترتیب توجه شود.

Expression's evaluation rules

$$\begin{array}{c}
\frac{\text{Fn}; \text{Env} \vdash e \rightarrow e'}{\text{Fn}; \text{Env} \vdash (e) \rightarrow (e')} [\text{E-Parenthesized-Operand}] \\
\\
\frac{\text{Fn}; \text{Env} \vdash e_1 \rightarrow e'_1}{\text{Fn}; \text{Env} \vdash e_1 \text{ bin_op } e_2 \rightarrow e'_1 \text{ bin_op } e_2} [\text{E-Binary-Left-Operand}] \\
\\
\frac{\text{Fn}; \text{Env} \vdash e_2 \rightarrow e'_2}{\text{Fn}; \text{Env} \vdash e_1 \text{ bin_op } e_2 \rightarrow e_1 \text{ bin_op } e'_2} [\text{E-Binary-Right-Operand}] \\
\\
\frac{\text{Fn}; \text{Env} \vdash e \rightarrow e'}{\text{Fn}; \text{Env} \vdash \text{uni_op } e \rightarrow \text{uni_op } e'} [\text{E-Unary-Operand}] \\
\\
\frac{\text{Fn}; \text{Env} \vdash e \rightarrow e'}{\text{Fn}; \text{Env} \vdash \text{id}(e) \rightarrow \text{id}(e')} [\text{E-Function-Call-Argument}] \\
\\
\frac{\text{id} \in \text{dom}(\text{Env})}{\text{Fn}; \text{Env} \vdash \text{Env}(\text{id})} [\text{E-Identifier}] \\
\\
\frac{}{\text{Fn}; \text{Env} \vdash (e) \rightarrow e} [\text{E-Parenthesized}] \\
\\
\frac{}{\text{Fn}; \text{Env} \vdash e_1 \text{ bin_op } e_2 \rightarrow \text{binary_eval}(\text{bin_op}, e_1, e_2)} [\text{E-Binary}] \\
\\
\frac{}{\text{Fn}; \text{Env} \vdash \text{un_op } e \rightarrow \text{unary_eval}(\text{un_op}, e)} [\text{E-Unary}] \\
\\
\frac{\text{Fn}(\text{id}_1) = (\text{id}_2, s) \quad \text{Fn}; \text{Env}, \text{id}_2 : e \vdash s \rightarrow s'}{\text{Fn}; \text{id}_2 : e \vdash \text{id}(e) \rightarrow s'} [\text{E-Function-Call}]
\end{array}$$

شکل ۶.۳: قاعده‌های ارزیابی عبارتها

با توجه به شکل ۷.۳ که قاعده‌های ارزیابی حکم‌ها را نشان می‌دهد، ارزیابی انواع حکم‌های توصیف شده در بخش قبلی، به صورت زیر است:

- حکمی که در یک حکم مرکب وجود دارد، در یک قدم باید ارزیابی شود.
- وقتی دنباله‌ای از حکم‌ها وجود دارد، تمام حکم‌های دنباله باید ارزیابی شوند.
- در صورتی که شرط یک حکم مقدار درست باشد، به حکم اولش در یک قدم ارزیابی شده و اگر شرط یک حکم مقدار غلط باشد، به حکم دومش در یک قدم ارزیابی می‌شود.
- شرط حکم شرطی باید در یک قدم ارزیابی شود.

- یک حلقه در یک قدم باید ارزیابی شده و با یک شرط و ادامه حلقه جایگزین شود تا در صورت درست بودن شرط حلقه، یکبار بدنه آن اجرا شده و در ادامه دوباره حلقه ارزیابی شود.
- حکمی که فقط شامل یک عبارت است، باید در یک قدم عبارتش ارزیابی شود.
- عبارتی که به یک متغیر تخصیص داده می‌شود باید در یک قدم ارزیابی شده و در محیط به‌عنوان مقدار فعلی متغیر ذخیره شود.
- عبارتی که بازگردانده می‌شود، باید در یک قدم ارزیابی شود.

Statement's evaluation rules

$$\begin{array}{c}
\frac{\text{Fn; Env} \vdash s \longrightarrow s'}{\text{Fn; Env} \vdash \{s\} \longrightarrow \{s'\}} \text{[E-Compound-Statement]} \\
\\
\frac{\text{Fn; Env} \vdash s_1 \longrightarrow s_1'}{\text{Fn; Env} \vdash s_1 \ s_2 \longrightarrow s_1' \ s_2} \text{[E-Sequence-Left-Statement]} \\
\\
\frac{\text{Fn; Env} \vdash s_2 \longrightarrow s_2'}{\text{Fn; Env} \vdash s_1 \ s_2 \longrightarrow s_1 \ s_2'} \text{[E-Sequence-Right-Statement]} \\
\\
\frac{}{\text{Fn; Env} \vdash \text{if (true) } s_1 \text{ else } s_2 \longrightarrow s_1} \text{[E-If-Statement-True]} \\
\\
\frac{}{\text{Fn; Env} \vdash \text{if (false) } s_1 \text{ else } s_2 \longrightarrow s_2} \text{[E-If-Statement-False]} \\
\\
\frac{\text{Fn; Env} \vdash e \longrightarrow e'}{\text{Fn; Env} \vdash \text{if (e) } s_1 \text{ else } s_2 \longrightarrow \text{if (e')} s_1 \text{ else } s_2} \text{[E-If-Statement]} \\
\\
\frac{}{\text{Fn; Env} \vdash \text{while (e) } s \longrightarrow \text{if (e) } \{ s \text{ while (e) } s \} \text{ else ;}} \text{[E-While-Statement]} \\
\\
\frac{\text{Fn; Env} \vdash e \longrightarrow e'}{\text{Fn; Env} \vdash e; \longrightarrow e';} \text{[E-Expression-Statement]} \\
\\
\frac{\text{Fn; Env} \vdash e \longrightarrow e'}{\text{Fn; Env} \vdash \text{id} = e; \longrightarrow \text{id} = e';, \text{Env}[\text{id} \mapsto e']} \text{[E-Assignment]} \\
\\
\frac{\text{Fn; Env} \vdash e \longrightarrow e'}{\text{Fn; Env} \vdash \text{return } e; \longrightarrow \text{return } e';} \text{[E-Return-Statement]}
\end{array}$$

شکل ۷.۳: قاعده‌های ارزیابی جمله‌ها

با توجه به شکل ۸.۳ که قاعده‌های ارزیابی برنامه را نشان می‌دهد، یک برنامه شامل دنباله‌ای از توابع بوده و هربار که یک تابع ارزیابی می‌شود، باید با توجه به شناسه‌اش، بدنه و ورودی‌اش در مجموعه توابع قرار بگیرند.

Program's evaluation rules

$$\frac{\text{Fn}; \text{Env} \vdash p \longrightarrow p'}{\text{Fn}; \text{Env} \vdash T_1 \text{ id}_1 (T_2 \text{ id}_2) s \ p \longrightarrow T_1 \text{ id}_1 (T_2 \text{ id}_2) s \ p', \text{Fn}[\text{id}_1 \mapsto (\text{id}_2, s)]} \text{[E-Function-Declaration]}$$

شکل ۸.۳: قاعده‌های ارزیابی برنامه

۴.۳ قاعده‌های نوع‌دهی

قاعده‌های نوع‌دهی قوانینی هستند که مشخص می‌کنند هر عبارت دارای چه نوعی بوده و چگونه می‌توان از منابع و متغیرها در طول برنامه استفاده نمود. این قواعد درستی برنامه از نظر نوع‌دهی و خوش نوع بودن آن را نشان می‌دهند. در صورتی که یک برنامه با استفاده از این قاعده‌ها قابل نوع‌دهی نباشد دارای خطا بوده و باید اصلاح شود. به منظور توصیف قاعده‌های نوع‌دهی برای زبان Linear-C ابتدا نیاز است تا دو مفهوم زمینه نوع و زمینه متغیرهای خطی تعریف شوند. زمینه نوع شامل زوج‌های تشکیل شده از یک شناسه و نوع آن شناسه است. زمینه متغیرهای خطی نیز یک مجموعه از شناسه‌هایی است که دارای نوع خطی بوده و صاحب^{۱۳} یک مقدار هستند. تعریف این دو مفهوم به منظور نوع‌دهی به عبارت‌های زبان Linear-C ضروری بوده و قوانین نوع خطی در این قاعده‌ها اعمال می‌شوند. شکل ۹.۳ توصیف زمینه نوع و زمینه متغیرهای خطی را نشان می‌دهد.

<i>Type context's definition</i>		<i>Linear context's definition</i>	
$\Gamma ::=$	type context:	$\Delta ::=$	linear context:
$\Gamma, \text{id} : \text{Type}$	type binding	Δ, id	owner
ε	empty type context	ε	empty linear context

شکل ۹.۳: توصیف زمینه نوع و زمینه متغیرهای خطی

^{۱۳} owner

با توجه به شکل ۱۰.۳ که قاعده‌های نوع‌دهی عملیات‌ها را نشان می‌دهد، عملیات‌هایی که در بخش‌های قبلی توصیف شدند به صورت زیر نوع‌دهی خواهند شد:

- عملیات جمع فقط بین دو عبارت با نوع عدد صحیح قابل انجام بوده و نتیجه یک عدد صحیح است.
- عملیات‌های عطف منطقی، فصل منطقی، برابری، کوچکتر بودن و بزرگتر بودن فقط بر روی دو عبارت با نوع بولی قابل انجام بوده و نتیجه نوع بولی خواهد داشت.
- در صورتی که یکی از عبارت‌های استفاده شده در یک عملیات دوگانی، دارای نوع خطی باشد، می‌توان مقدار آن را قرض گرفته و به عنوان عبارتی با نوع غیرخطی با آن برخورد نمود. در این صورت، مقدار آن مصرف نشده و مشکلی ایجاد نمی‌شود.
- عملیات منفی کردن فقط بر روی عبارت‌هایی با نوع عدد صحیح قابل انجام بوده و نتیجه یک عدد صحیح است.
- عملیات تکیض فقط بر روی یک عبارت با نوع بولی قابل انجام بوده و نتیجه دارای نوع بولی است.
- عملگر آدرس در صورتی که بر روی یک عبارت اعمال شود، نتیجه دارای اشاره‌گری به نوع عبارت موردنظر است.
- عملگر در آدرس اگر بر روی یک عبارت با نوع اشاره‌گر به یک نوع دیگر اعمال شود، نتیجه دارای آن نوع دیگر است.
- در صورتی که عبارت استفاده شده در یک عملیات یگانی دارای نوع خطی باشد، می‌توان مقدار آن را قرض گرفته و به عنوان عبارتی با نوع غیرخطی با آن برخورد نمود. در این صورت، مقدار آن مصرف نشده و مشکلی ایجاد نمی‌شود.

Binary operation's typing rules

$$\begin{aligned} \text{binary}(+, \text{Int}, \text{Int}) &: \text{Int} & \text{binary}(\&\&, \text{Bool}, \text{Bool}) &: \text{Bool} & \text{binary}(\|, \text{Bool}, \text{Bool}) &: \text{Bool} \\ \text{binary}(==, \text{Bool}, \text{Bool}) &: \text{Bool} & \text{binary}(<, \text{Bool}, \text{Bool}) &: \text{Bool} & \text{binary}(>, \text{Bool}, \text{Bool}) &: \text{Bool} \\ \text{binary}(\text{bin_op}, \text{Linear } T, T) &: \text{binary}(\text{bin_op}, T, T) \\ \text{binary}(\text{bin_op}, T, \text{Linear } T) &: \text{binary}(\text{bin_op}, T, T) \end{aligned}$$

Unary operation's typing rules

$$\begin{aligned} \text{unary}(-, \text{Int}) &: \text{Int} & \text{unary}(!, \text{Bool}) &: \text{Bool} \\ \text{unary}(\&, T) &: T^* & \text{unary}(*, T^*) &: T \\ \text{unary}(\text{uni_op}, \text{Linear } T) &: \text{unary}(\text{uni_op}, T) \end{aligned}$$

با توجه به شکل ۱۱.۳ که قاعده‌های نوع‌دهی عبارت‌ها را نشان می‌دهد، عبارت‌هایی که در بخش‌های قبلی توصیف شدند به صورت زیر نوع‌دهی خواهند شد:

- عبارت پراتنژدار، نوعش همان نوع عبارت داخل پراتنژ است.
- در صورتی که یک عملیات دوگانی بین دو عبارت انجام شود، نوع نتیجه با استفاده از قاعده‌های نوع‌دهی عملیات‌ها مشخص می‌شود.
- در صورتی که یک عملیات یگانی بر روی یک عبارت انجام شود، نوع نتیجه با استفاده از قاعده‌های نوع‌دهی عملیات‌ها مشخص می‌شود.
- در صورتی که یک شناسه در زمینه نوع وجود داشته باشد، نوعش از آن استخراج شده و مشخص می‌شود.
- اگر تابعی دارای ورودی با نوع خطی باشد و یک متغیر با نوع خطی به آن داده شود، این شناسه از زمینه متغیرهای خطی حذف می‌شود زیرا مقدار آن منتقل شده و دیگر شناسه مورد نظر صاحب مقدار نیست. همچنین، نتیجه این فراخوانی دارای نوع بازگردانده شده توسط تابع است.
- اگر تابعی دارای ورودی با نوع غیرخطی باشد و یک متغیر با نوع خطی به آن داده شود، مقدار این متغیر قرض گرفته شده و شناسه آن از زمینه متغیرهای خطی حذف نمی‌شود، همچنین، نتیجه این فراخوانی دارای نوع بازگردانده شده توسط تابع است.
- اگر تابعی دارای ورودی با نوع غیرخطی باشد و یک متغیر با نوع غیرخطی به آن داده شود، نتیجه این فراخوانی دارای نوع بازگردانده شده توسط تابع است.

با توجه به شکل ۱۲.۳ که قاعده‌های نوع‌دهی حکم‌ها را نشان می‌دهد، حکم‌هایی که در بخش‌های قبلی توصیف شدند به صورت زیر نوع‌دهی خواهند شد:

- یک حکم مرکب، نوع حکمی که در داخل خود دارد را خواهد داشت.
- نوع دنباله‌ای از حکم‌ها، نوع آخرین حکم در این دنباله است.
- در صورتی که یک حکم شرطی وجود داشته باشد، نوع آن با نوع دو حکمی که در خود دارد یکسان خواهد بود.
- نوع یک حلقه با نوع حکمی که در داخل خود دارد، یکسان است.
- نوع یک حکم شامل عبارت، نوع واحد است.
- در صورتی که شناسه استفاده شده در تعریف متغیر قبلاً استفاده نشده باشد، نوع آن نوع واحد خواهد بود.
- در صورتی که یک متغیر با نوع خطی به یک متغیر دیگر با نوع خطی تخصیص داده شود، شناسه متغیر سمت راست از زمینه متغیرهای خطی حذف شده و شناسه متغیر سمت چپ به این زمینه اضافه می‌شود زیرا متغیر سمت راست مقدارش به متغیر سمت چپ منتقل شده است، همچنین، متغیر سمت چپ نباید از قبل مقداری داشته باشد زیرا در این صورت، مقدار قبلیش رونوشت^{۱۴} شده و نشتی حافظه رخ خواهد داد. نوع این حکم نیز نوع واحد است.

^{۱۴} overwrite

- در صورتی که یک متغیر با نوع خطی به یک متغیر با نوع غیرخطی تخصیص داده شود، اگر متغیر سمت راست در زمینه متغیرهای خطی وجود داشته و صاحب مقدار باشد، مقدارش به متغیر سمت چپ قرض داده شده و حکم دارای نوع واحد است.
- اگر یک عبارت با نوع خطی به یک متغیر با نوع غیرخطی تخصیص داده شود، شناسه متغیر سمت چپ به زمینه متغیرهای خطی اضافه می‌شود، همچنین، متغیر سمت چپ نباید از قبل مقداری داشته زیرا در این صورت، مقدار قبلیش رونوشت شده و نشستی حافظه رخ خواهد داد. نوع این حکم نیز نوع واحد است.
- اگر یک عبارت به یک متغیر نسبت داده شود، حکم دارای نوع واحد خواهد بود.
- در صورتی که متغیری دارای مقدار نبوده و زمینه متغیرهای خطی خالی باشد، می‌توان از تابع بازگشت که این حکم دارای نوع واحد است.
- حکم یک نقطه ویرگول دارای نوع واحد است.

Expression's typing rules

$$\begin{array}{c}
 \frac{\Gamma; \Delta \vdash e : T}{\Gamma; \Delta \vdash (e) : T} [\text{T-Parenthesized}] \\
 \\
 \frac{\Gamma; \Delta \vdash e_1 : T_1 \quad \Gamma; \Delta \vdash e_2 : T_2 \quad \Gamma; \Delta \vdash \text{binary}(\text{bin_op}, T_1, T_2) : T_3}{\Gamma; \Delta \vdash e_1 \text{ bin_op } e_2 : T_3} [\text{T-Binary}] \\
 \\
 \frac{\Gamma; \Delta \vdash e : T \quad \Gamma; \Delta \vdash \text{unary}(\text{un_op}, T) : T'}{\Gamma; \Delta \vdash \text{uni_op } e : T'} [\text{T-Unary}] \\
 \\
 \frac{\Gamma; \Delta \vdash v : T}{\Gamma; \Delta \vdash v : T} [\text{T-Value}] \\
 \\
 \frac{\Gamma(\text{id}_1) = \text{Linear } T \rightarrow T' \quad \Gamma(\text{id}_2) = \text{Linear } T}{\Gamma; \Delta \vdash \text{id}_1(\text{id}_2) : T', \Delta \setminus \{\text{id}_2\}} [\text{T-Function-Call-Move}] \\
 \\
 \frac{\Gamma(\text{id}_1) = T \rightarrow T' \quad \Gamma(\text{id}_2) = \text{Linear } T}{\Gamma; \Delta \vdash \text{id}_1(\text{id}_2) : T'} [\text{T-Function-Call-Burrow}] \\
 \\
 \frac{\Gamma(\text{id}) = \text{Linear } T \rightarrow T' \quad \Gamma; \Delta \vdash e : T}{\Gamma; \Delta \vdash \text{id}(e) : T'} [\text{T-Function-Call}]
 \end{array}$$

شکل ۱۱.۳: قاعده‌های نوع‌دهی عبارتها

Statement's typing rules

$$\frac{\Gamma, \Gamma'; \Delta \vdash s : T}{\Gamma; \Delta \vdash \{s\} : T} [\text{T-Compound-Statement}]$$

$$\frac{\Gamma; \Delta \vdash s_1 : T_1 \quad \Gamma; \Delta \vdash s_2 : T_2}{\Gamma; \Delta \vdash s_1 \ s_2 : T_2} [\text{T-Sequence}]$$

$$\frac{\Gamma; \Delta \vdash s_1 : T \quad \Gamma; \Delta \vdash s_2 : T \quad \Gamma; \Delta \vdash e : \text{Bool}}{\Gamma; \Delta \vdash \text{if } (e) \ s_1 \ \text{else } s_2 : T} [\text{T-If-Statement}]$$

$$\frac{\Gamma; \Delta \vdash e : \text{Bool} \quad \Gamma; \Delta \vdash s : T}{\Gamma; \Delta \vdash \text{while } (e) \ s : T} [\text{T-While-Statement}]$$

$$\frac{\Gamma; \Delta \vdash e : \text{Unit}}{\Gamma; \Delta \vdash e; : \text{Unit}} [\text{T-Expression-Statement}]$$

$$\frac{\text{id} \notin \text{dom}(\Gamma)}{\Gamma; \Delta \vdash T \ \text{id}; : \text{Unit}, \Gamma[\text{id} \mapsto T]} [\text{T-Declaration}]$$

$$\frac{\Gamma(\text{id}_1) = \text{Linear } T \quad \Gamma(\text{id}_2) = \text{Linear } T \quad \text{id}_1 \notin \Delta \quad \text{id}_2 \in \Delta}{\Gamma; \Delta \vdash \text{id}_1 = \text{id}_2; : \text{Unit}, (\Delta \setminus \{\text{id}_2\}) \cup \{\text{id}_1\}} [\text{T-Assignment-Move}]$$

$$\frac{\Gamma(\text{id}_1) = T \quad \text{id}_2 \in \Delta \quad \Gamma(\text{id}_2) = \text{Linear } T}{\Gamma; \Delta \vdash \text{id}_1 = \text{id}_2; : \text{Unit}} [\text{T-Assignment-Burrow}]$$

$$\frac{\Gamma(\text{id}) = \text{Linear } T \quad \text{id} \notin \Delta \quad \Gamma; \Delta \vdash e : \text{Linear } T}{\Gamma; \Delta \vdash \text{id} = e; : \text{Unit}, \Delta \cup \text{id}} [\text{T-Assignment-Linear}]$$

$$\frac{\Gamma(\text{id}) = T \quad \Gamma; \Delta \vdash e : T}{\Gamma; \Delta \vdash \text{id} = e; : \text{Unit}} [\text{T-Assignment}]$$

$$\frac{}{\Gamma; \emptyset \vdash \text{return } e; : \text{Unit}} [\text{T-Return-Statement}]$$

$$\frac{}{\Gamma; \Delta \vdash ; : \text{Unit}} [\text{T-Semicolon-Statement}]$$

شکل ۱۲.۳: قاعده‌های نوع‌دهی حکم‌ها

با توجه به شکل ۱۳.۳ که قاعده‌های نوع‌دهی برنامه را نشان می‌دهد، برنامه‌ها بر اساس توصیف‌های انجام شده در بخش‌های قبل به صورت زیر نوع‌دهی خواهند شد:

- اگر تابعی با ورودی دارای نوع خطی تعریف شود، نوع بدنه تابع با زمینه متغیرهای خطی تشکیل شده از شناسه ورودی تابع تعیین شده و نتیجه دارای نوع بدنه تابع خواهد بود.
- اگر تابعی تعریف شود، نوع بدنه تابع تعیین شده و نتیجه دارای نوع بدنه تابع خواهد بود.
- برنامه خالی دارای نوع واحد است.

Program's typing rules

$$\frac{id_1 \notin \text{dom}(\Gamma) \quad \Gamma, id_1 : T_1, id_2 : T_2; \{id_2\} \vdash s : T_3}{\Gamma; \emptyset \vdash T_1 \quad id_1(\text{Linear } T_2 \quad id_2) \quad s : T_3, \Gamma[id \mapsto T_1 \rightarrow T_2]} [\text{T-Function-Declaration-Linear-Argument}]$$

$$\frac{id_1 \notin \text{dom}(\Gamma) \quad \Gamma, id_1 : T_1, id_2 : T_2; \emptyset \vdash s : T_3}{\Gamma; \emptyset \vdash T_1 \quad id_1(T_2 \quad id_2) \quad s : T_3, \Gamma[id \mapsto T_1 \rightarrow T_2]} [\text{T-Function-Declaration}]$$

$$\frac{}{\Gamma; \emptyset \vdash \varepsilon : \text{Unit}} [\text{T-Empty-Program}]$$

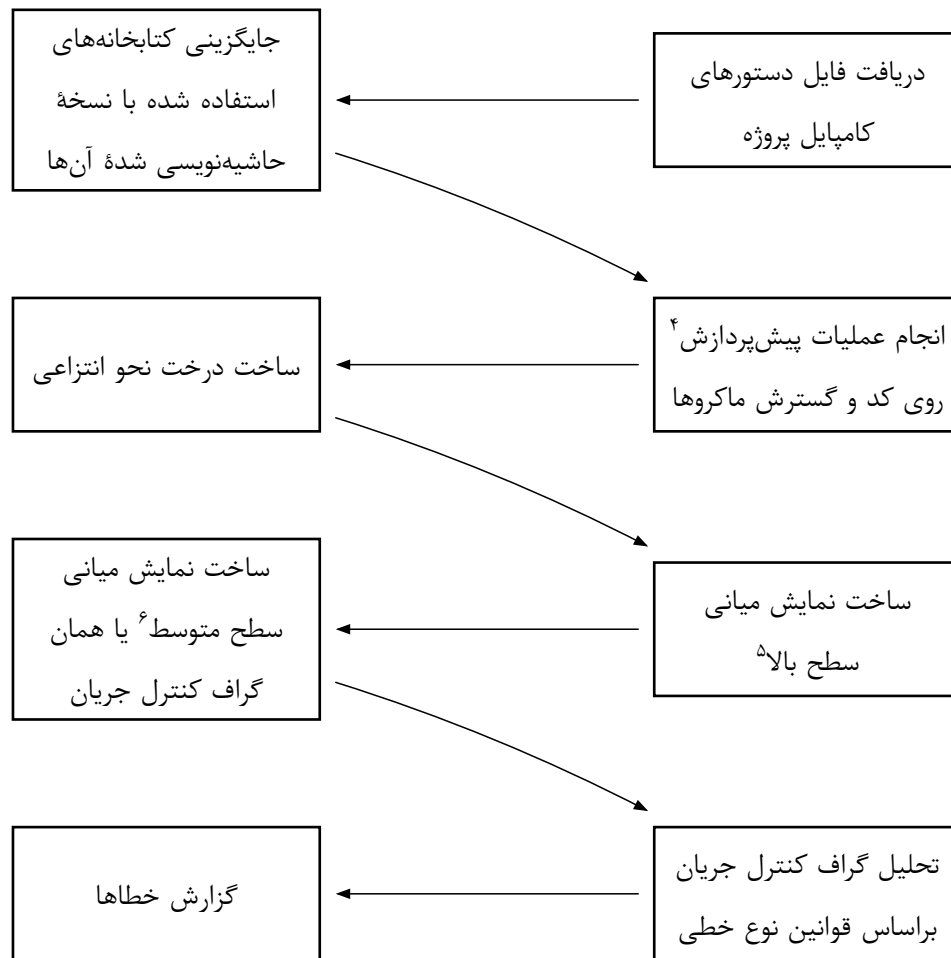
شکل ۱۳.۳: قاعده‌های نوع‌دهی برنامه

فصل چهارم

پیاده‌سازی و ارزیابی

۱.۴ اعمال قوانین نوع خطی

ابتدا نیاز است تا یک پیاده‌سازی از سامانه نوع پیشنهادی صورت بگیرد. شکل ۱.۴ نمودار بلوکی^۱ ابزار پیاده‌سازی شده را نشان می‌دهد. بخش‌های اصلی این پیاده‌سازی شامل مراحل پیش‌پردازش، تشکیل درخت نحو انتزاعی^۲ و تشکیل گراف کنترل جریان^۳ است. در ادامه، ابزار تحلیلی براساس قوانین نوع خطی بر روی گراف کنترل جریان برنامه انجام داده و خطاهای پیش آمده را گزارش می‌کند. پیاده‌سازی ابزار در مخزن [۷] قرار دارد.



شکل ۱.۴: نمودار بلوکی ابزار

^۱ block diagram

^۲ abstract syntax tree

^۳ control flow graph

^۴ preprocessing

^۵ HIR - high-level intermediate representation

^۶ MIR - mid-level intermediate representation

با توجه به شکل ۱.۴، ابتدا فایل دستوره‌های کامپایل پروژه از کاربر دریافت شده که ساختار پروژه را مشخص می‌کند. سپس، کتابخانه‌ها با نسخه حاشیه‌نویسی شده آن‌ها جایگزین شده و عملیات پیش‌پردازش به‌منظور گسترش ماکروها بر روی کد صورت می‌گیرد. نحو زبان برنامه‌نویسی C برای ابزار تری سیت^۷ توصیف شده و پارسر^۸ زبان توسط این ابزار تولید شده است. با استفاده از پارسر تولید شده، درخت نحو انتزاعی برنامه ورودی تهیه شده و نمایش میانی سطح بالا از روی این درخت ساخته می‌شود. نمایش میانی سطح بالا نمایش ساده شده^۹ کد می‌باشد. در ادامه نمایش میانی سطح متوسط و یا همان گراف کنترل جریان برنامه ساخته می‌شود.

گراف کنترل جریان ابزاری بنیادی در تحلیل‌های ایستا و پویا به شمار می‌رود که با نمایش تمامی مسیرهای اجرایی ممکن در برنامه، بستری مناسب برای بررسی ویژگی‌های معنایی کد فراهم می‌کند. دلیل استفاده از گراف کنترل جریان به‌منظور بررسی برقراری قوانین نوع خطی، نیاز به بررسی تمامی مسیرهای اجرایی برنامه می‌باشد. با بررسی تمام مسیرهای اجرایی برنامه، می‌توان مطمئن بود که برنامه با ایراد حافظه مواجه نخواهد شد. شکل ۳.۴ نشان‌دهنده نمایش گراف کنترل جریان تولید شده توسط ابزار برای قطعه کد شکل ۲.۴ است.

نوع داده اصلی در گراف کنترل جریان، بدنه^۹ است. یک بدنه نمایشی از یک تابع بوده و شامل تعدادی بلوک پایه^{۱۰} است. مهم‌ترین نوع داده‌های استفاده شده در گراف کنترل جریان، به‌صورت زیر هستند:

- بدنه: نمایشی از بدنه یک تابع بوده و شامل تعدادی بلوک پایه است.
- بلوک پایه: بلوک‌های پایه دربرگیرنده تعدادی حکم^{۱۱} هستند که در آن‌ها هیچ شاخه‌ای وجود ندارد. تنها جایی که ممکن است شاخه وجود داشته باشد، در بخش خروجی بلوک پایه است.
- حکم: حکم‌ها نشان‌دهنده عملیات‌های ساده‌ای اعم از اختصاص دادن مقدار یا متغیر هستند.
- پایان‌دهنده^{۱۲}: پایان‌دهنده‌ها نشان‌دهنده شاخه بوده و می‌توانند در انتهای یک بلوک پایه وجود داشته باشند.

^۷ tree-sitter

^۸ parser

^۹ body

^{۱۰} basic block

^{۱۱} statement

^{۱۲} terminator

```

main.c

void main() {
    int i = 1 + 2 + 3;
    if (i < 10) {
        i = 10;
    } else {
        i = 100;
    }
    return;
}

```

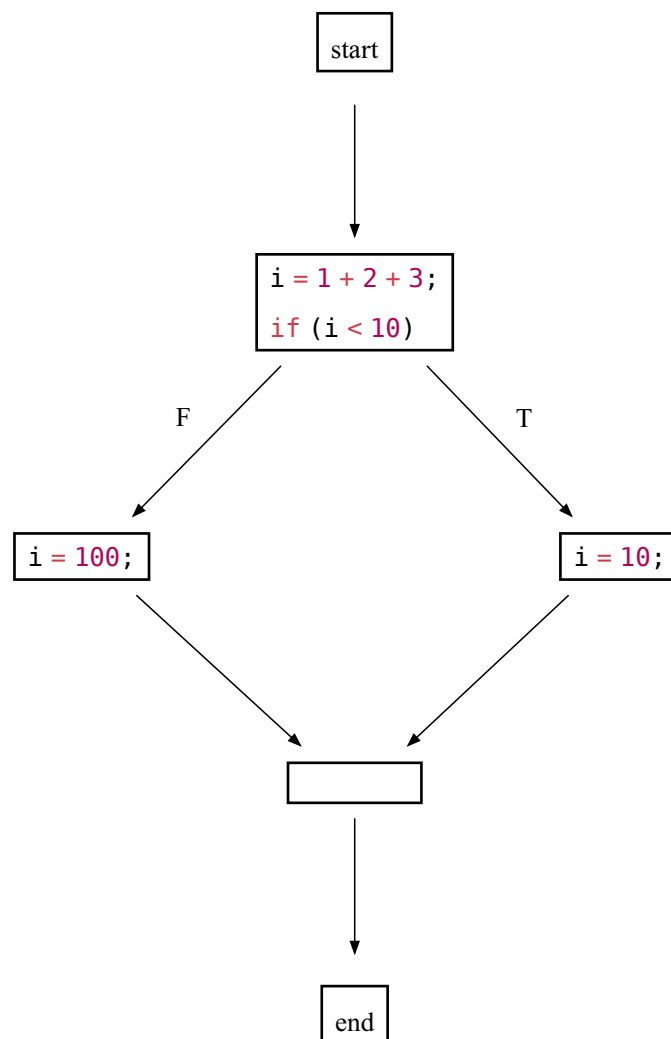
شکل ۲.۴: قطعه کد ساده دارای شاخه

<pre> main.c let main_0: void; let _1: temp; let i_2: int; let _3: temp; 'bb_0: { _1 = 1 + 2; i_2 = _1 + 3; _3 = i_2 < 10; switch _3 { 1 => 'bb_1; _ => 'bb_2; } } </pre>	<pre> 'bb_1: { i_2 = 10; goto 'bb_3; } 'bb_2: { i_2 = 100; goto 'bb_3; } 'bb_3: { return; } </pre>
---	---

شکل ۳.۴: نمایش گراف کنترل جریان برای یک قطعه کد ساده توسط ابزار

گراف کنترل جریان نمایش داده شده توسط ابزار با نمایش مرسوم گراف کنترل جریان متفاوت است. این تفاوت به دلیل اختصاص یک شناسه یکتا به متغیرها به منظور ایجاد تمایز بین متغیرهای هم‌نام، استفاده از متغیرهای کمکی به منظور ساده‌سازی عبارت‌ها و دیگر عملیات‌های انجام شده در مراحل قبلی است. با توجه به موارد بیان شده، شکل ۴.۴ نشان‌دهنده نمایش مرسوم گراف کنترل جریان برای قطعه کد شکل ۲.۴ است. همانطور که مشاهده می‌شود، به منظور ساده‌سازی عملیات جمع از یک متغیر کمکی استفاده شده و متغیر تعریف شده دارای یک شناسه یکتا در کنار نامش است.

با ساخت گراف کنترل جریان برای کد ورودی، می‌توان مسئله برقراری قوانین نوع خطی را به یک مسئله مشخص بر روی گراف‌ها تبدیل نموده و سپس با استفاده از الگوریتم‌های مربوط به گراف‌ها، مسئله را حل نمود. در ادامه، مسئله به طور دقیق‌تر تعریف شده و سپس الگوریتم پیشنهادی به منظور تشخیص برقراری قوانین نوع خطی بررسی خواهد شد. این الگوریتم نشان می‌دهد که چگونه می‌توان از نظریه‌ی گراف‌ها برای حل چنین چالش‌هایی در طراحی و تحلیل زبان‌های برنامه‌نویسی استفاده نمود.



شکل ۴.۴: نمایش مرسوم گراف کنترل جریان برای یک قطعه کد ساده

۲.۴ الگوریتم پیشنهادی

مسئله وجود خطای حافظه را می‌توان بر روی گراف کنترل جریان اینگونه توصیف کرد: اگر با داشتن وضعیت یک متغیر دارای نوع خطی، از یک گره مشخص جست‌وجو آغاز شده و در ادامه، گره‌ای پیدا شود که نشان‌دهنده استفاده نامعتبر از آن متغیر باشد آنگاه می‌توان نتیجه گرفت که یک خطای حافظه رخ داده و لازم است تا این خطا گزارش شود. در اینجا منظور از استفاده نامعتبر این است که از متغیری که مقدارش منتقل شده است استفاده شود و یا به متغیری که هنوز مقدارش را منتقل نکرده است، مقدار جدیدی اختصاص داده شود. با در نظر گرفتن این تعریف مسئله، می‌توان الگوریتمی طراحی نمود که برقراری قوانین نوع خطی در برنامه را بررسی کرده و با استفاده از گراف کنترل جریان کد، تحلیل‌های مناسب را انجام دهد.

به منظور حل مسئله برقراری قوانین نوع خطی و بررسی وجود خطای حافظه در کد، ابتدا متغیرهایی که دارای نوع خطی هستند شناسایی شده و سپس به ازای هر متغیر، با شروع از تک تک گره‌های گراف، جست‌جوی عمق اول^{۱۳} و یا جست‌جوی سطح اول^{۱۴} انجام می‌شود. هر جست‌وجو تا گره‌ای ادامه پیدا می‌کند که وضعیت متغیر را تغییر دهد. در این صورت، اگر استفاده نامعتبری از متغیر وجود داشته باشد، پیدا شده و گزارش داده می‌شود. این الگوریتم برای بررسی یک تابع در صورتی که N تعداد متغیرهای محلی دارای نوع خطی، S تعداد حکم‌های تابع و V تعداد بلوک‌های پایه تابع باشد دارای پیچیدگی^{۱۵} زمانی از مرتبه $O(NVS)$ است. همچنین، اگر M تعداد توابع برنامه باشد، پیچیدگی زمانی برنامه از مرتبه $O(MNVS)$ خواهد بود. پس از پیاده‌سازی الگوریتم توصیف شده، نیاز است تا درستی و دقت ابزار سنجیده شود. در ادامه، آزمایش‌های انجام شده به منظور سنجش و ارزیابی ابزار بیان شده و نتایج آن‌ها مورد بررسی قرار گرفته است.

۳.۴ آزمایش‌ها

برای سنجش درستی و کامل بودن ابزار، ابتدا لازم است تا محیطی مناسب برای اجرای آزمایش‌ها و ارزیابی فراهم گردد. به منظور سنجش ابزار باید نمونه‌های آزمایشی تهیه شود تا ابزار تحلیل را بر روی آن انجام داده و باتوجه به خروجی و مقایسه آن با خروجی مورد انتظار، دقت و کارایی ابزار ارزیابی شود. این نمونه‌های آزمایشی باید شامل توابع با ساختارهای متفاوت، استفاده‌های گوناگون از متغیرهای خطی و شرایط پیچیده کنترل جریان باشند به گونه‌ای که بتوان نقاط قوت و ضعف سامانه را به طور کامل شناسایی نمود.

^{۱۳} depth-first search

^{۱۴} breadth-first search

^{۱۵} time complexity

پس از آماده‌سازی نمونه‌ها و محیط آزمایش، آزمایش‌های اولیه‌ای باید صورت بگیرند تا درستی عملکرد محیط آزمایش تأیید گردد. در این مرحله بررسی می‌شود که ابزار به درستی اجرا شده و گزارش‌ها به شکل دقیق و کامل تولید می‌شوند و هیچ خطایی در فرایند ثبت یا اجرای نمونه‌ها وجود ندارد. نتایج این مرحله مبنایی برای اطمینان از آماده‌بودن محیط آزمایش جهت اجرای آزمایش‌های گسترده‌تر و جامع‌تر هستند. در نهایت محیط آزمایش به گونه‌ای تنظیم می‌شود که امکان انجام آزمایش‌های کامل و قابل مقایسه فراهم گردیده و داده‌های حاصل بتوانند برای ارزیابی عملکرد ابزار، مورد استفاده قرار بگیرند. این آماده‌سازی دقیق، پایه‌ای مطمئن برای تحلیل کامل و سنجش کارایی سامانه فراهم کرده و اطمینان می‌دهد که نتایج به دست‌آمده نمایانگر واقعیت عملکرد ابزار در شرایط مختلف هستند.

نمونه‌های آزمایشی موردنیاز به‌منظور ارزیابی و سنجش ابزار، با استفاده از مدل‌های زبانی بزرگ^{۱۶} تولید شده‌اند و شامل ۲۲۰ برنامه مختلف نوشته شده با استفاده از زبان Linear-C هستند. از این برنامه‌ها، ۱۰۰ برنامه دارای خطای حافظه بوده و ۱۲۰ برنامه ایمن هستند. شکل ۵.۴ دستورهای استفاده شده به‌منظور تولید نمونه‌های آزمایشی با استفاده از مدل‌های زبانی بزرگ را نشان می‌دهد.

Consider these memory safety issues: use-after-free, memory-leak, double-free, dangling-pointer. Try to generate 100 C codes based on these which have memory issues. The scenarios must have function definition and function calling. Don't use struct, enum, and union, and global variables. Name the files as case-unsafe-1.c, ... Remove the comments and make them as diverse as possible. Don't copy the examples just learn the patterns and combine them in complicated scenarios. They must at least have three functions defined.

Consider these memory safety issues: use-after-free, memory-leak, double-free, dangling-pointer. Try to generate 120 C codes based on these which do not have memory issues. The scenarios must have function definition and function calling. Don't use struct, enum, and union, and global variables. Name the files as case-safe-1.c, ... Remove the comments and make them as diverse as possible. Don't copy the examples just learn the patterns and combine them in complicated scenarios. They must at least have three functions defined.

شکل ۵.۴: دستورهای استفاده شده به‌منظور تولید نمونه‌های آزمایشی

^{۱۶} large language model

نمونه‌های تولید شده قابل کامپایل بوده و کامپایلر زبان برنامه‌نویسی C خطایی برای وجود ایراد حافظه در آن‌ها نمی‌دهد. شکل ۶.۴ یک نمونه از برنامه‌های تولید شده که دارای حافظه است را نشان می‌دهد. این برنامه دارای ایراد دسترسی به حافظه پس از آزادسازی بوده و کامپایلر زبان برنامه‌نویسی C ایرادی از آن نمی‌گیرد ولی ابزار نوشته شده این خطا را تشخیص می‌دهد.

با داشتن برچسب هر برنامه و ورودی دادن آن به ابزار، خروجی ابزار ذخیره شده و دقت و بازیابی^{۱۷} محاسبه شده است. جدول ۱.۴ نتیجه ارزیابی ابزار را نشان می‌دهد. شکل ۷.۴ نیز نمودار دایره‌ای^{۱۸} نتایج به دست آمده را نمایش می‌دهد. باتوجه به نتایج به دست آمده، میزان دقت و بازیابی ابزار به‌صورت زیر محاسبه می‌شود. از آنجایی که در نتایج نمونه منفی غلط وجود ندارد، می‌توان گفت که ابزار درست است. همچنین، با توجه به تعداد نسبتاً کم نمونه‌های مثبت غلط، ابزار از دقت مناسبی برخوردار بوده و می‌توان در عمل از آن استفاده نمود.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{100}{100 + 28} = 0.78125$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{100}{100 + 0} = 1$$

^{۱۷}precision and recall

^{۱۸}pie chart

```

main.c

#include <azhdaha.h>
#include <stdlib.h>

LINEAR_TYPE int *create_pattern(int length) {
    LINEAR_TYPE int *pattern = malloc(length * sizeof(int));
    for (int i = 0; i < length; i++) {
        pattern[i] = i * 4;
    }
    return pattern;
}

void transform_pattern(LINEAR_TYPE int *pattern, int length) {
    for (int i = 0; i < length; i++) {
        pattern[i] * 2 + 1;
    }
    free(pattern);
}

int find_pattern_value(LINEAR_TYPE int *pattern, int length, int target) {
    for (int i = 0; i < length; i++) {
        if (pattern[i] == target) {
            return i;
        }
    }
    return -1;
}

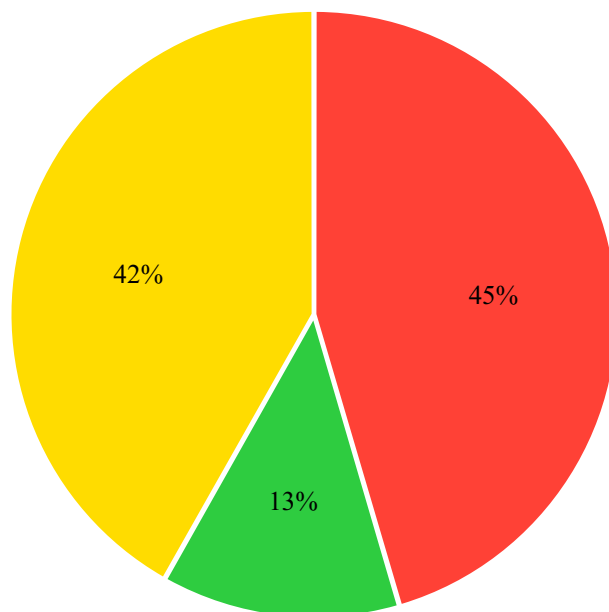
void main() {
    LINEAR_TYPE int *seq = create_pattern(30);
    transform_pattern(seq, 30);
    int pos = find_pattern_value(seq, 30, 15);
}

```

شکل ۶.۴: نمونه برنامه آزمایش شده که دارای ایراد حافظه است

خروجی ابزار			نسبت واقعی
برنامه دارای خطای حافظه است	برنامه بدون خطای حافظه است		
۱۰۰ مورد	۰ مورد	برنامه دارای خطای حافظه است	
۲۸ مورد	۹۲ مورد	برنامه بدون خطای حافظه است	

جدول ۱۰۴: نتیجه ارزیابی ابزار بر روی ۲۲۰ نمونه آزمایشی



منفی درست ■ مثبت غلط ■ منفی غلط ■ مثبت درست

شکل ۷.۴: نمودار دایره‌ای ارزیابی ابزار بر روی ۲۲۰ نمونه آزمایشی

فصل پنجم

جمع‌بندی و پیشنهادها

۱.۵ نتیجه‌گیری و جمع‌بندی

با وجود اینکه در سال‌های اخیر زبان‌های برنامه‌نویسی نوین با رویکردها و مدل‌های ایمنی حافظه جدیدی معرفی شده‌اند اما همچنان تعداد زیادی کتابخانه‌های سیستمی، چارچوب‌های پرکاربرد و نرم‌افزارهای پراستفاده نوشته شده با استفاده از زبان برنامه‌نویسی C وجود دارند که به‌طور گسترده استفاده می‌شوند. بازنویسی این پروژه‌ها با زبان‌های دیگر معمولاً پرهزینه بوده و در بسیاری از موارد از نظر اقتصادی و یا عملی به صرفه نیست. علاوه بر این، برخی از پروژه‌ها چنان با سامانه‌ها و کتابخانه‌های قدیمی در هم تنیده شده‌اند که جایگزینی کامل آن‌ها امکان‌پذیر نبوده و باعث از دست رفتن سازگاری می‌شود. بنابراین، نیاز به نگهداری، توسعه تدریجی و بهره‌برداری مستمر از این کدها همچنان باقی است.

این واقعیت، ضرورت نیاز به یک راهکار کم‌هزینه و کارآمد را نشان می‌دهد که بتواند از خطاهای مربوط به حافظه مانند نشت حافظه، استفاده از حافظه پس از آزادسازی و یا آزادسازی دوباره حافظه جلوگیری کرده و سرباری در زمان اجرای برنامه نداشته باشد. با توجه به اینکه یکی از دلایل اصلی استفاده از زبان برنامه‌نویسی C اهمیت سرعت بالا و عملکرد برنامه است پس چنین راهکاری باید دارای انتزاع بدون هزینه بوده و استفاده از آن هیچگونه هزینه و بار اضافی در زمان اجرای برنامه نداشته باشد تا به‌طور گسترده پذیرفته شده و مورد استفاده قرار بگیرد. با توجه به توضیحاتی که داد شده، استفاده از سامانه‌های نوع مانند سامانه نوع خطی، بسیار مفید بوده زیرا امکان تشخیص خطاهای حافظه و ناسازگاری‌ها را در زمان کامپایل فراهم می‌کنند و جلوی بسیاری از اشتباهات رایج مربوط به تخصیص و آزادسازی حافظه را می‌گیرند. با تعریف دقیق برای یک داده، کامپایلر می‌تواند بسیاری از خطاها را قبل از زمان اجرا شناسایی کرده و نیازی به آزمایش‌های پرهزینه در زمان اجرا نیست.

با به کارگیری این سامانه، فرآیند توسعه و نگهداری کد در طول زمان نیز ساده‌تر می‌شود زیرا توسعه‌دهندگان می‌توانند با اطمینان بیشتری تغییرهای مورد نیاز را اعمال کرده و از بروز خطاهای ناخواسته جلوگیری نمایند. این ویژگی در پروژه‌های تیمی اهمیت بیشتری پیدا می‌کند، زیرا افراد مختلف بدون ایجاد تداخل در کار یکدیگر می‌توانند به‌صورت همزمان بر روی بخش‌های گوناگون سیستم کار کنند. استفاده از چنین نوع‌هایی کمک می‌کند تا ساختار برنامه روشن‌تر و قابل‌فهم‌تر شده و مدیریت حافظه و منابع آسان‌تر گردد. در نتیجه، سامانه‌های نوع ابزار قدرتمندی برای تضمین ایمنی، ساماندهی و کنترل بهتر برنامه‌ها بوده و استفاده از آن‌ها در زبان‌هایی مانند زبان برنامه‌نویسی C که در آن‌ها دسترسی مستقیم بر روی حافظه وجود دارد، فواید بسیاری خواهد داشت.

۲.۵ چشم‌انداز و پیشنهادها

به‌منظور اضافه‌شدن نوع خطی به سامانه نوع زبان برنامه‌نویسی C، نیاز است تا سازوکاری برای استفاده از این نوع فراهم شود به‌طوری که کاربران بتوانند به راحت‌ترین شکل ممکن از آن استفاده نمایند. این سازوکار باید بتواند به پروژه‌های فعلی نوشته شده با استفاده از این زبان اضافه شده و با آن‌ها ادغام شود. همچنین، نیاز است تا کتابخانه‌های^۱ فعلی حاشیه‌نویسی^۲ شده و قوانین مربوط به نوع خطی بر روی آن‌ها نیز اعمال شود. نوع خطی باید طوری به پروژه اضافه شود که بر روی عملکرد فعلی برنامه هیچگونه تاثیری نگذارد. بدین‌منظور، می‌توان کتابخانه‌ای ارائه داد که تعریفی با عنوان تعریف عمومی ماکروی^۳ نوع خطی در آن قرار داشته باشد. تعریف عمومی ماکروی نوع خطی می‌تواند مانند تعریف آورده شده در شکل ۱.۵ باشد.

```
azhdaha.h

#define LINEAR_TYPE
```

شکل ۱.۵: تعریف عمومی ماکروی نوع خطی

این ماکرو در هنگام کامپایل شدن برنامه و در بخش پیش‌پردازش از بین رفته و چیزی در جایش باقی نمی‌ماند. درنتیجه، کد نوشته شده به حالت اولیه خود بازگشته و تاثیری از این کلیدواژه در برنامه نهایی باقی نخواهد ماند. کاربران با اضافه نمودن این کتابخانه به پروژه خود قادر خواهند بود تا از این ماکرو در تعریف‌هایشان استفاده نمایند. درنتیجه، استفاده از آن سخت نبوده و مانند یک کتابخانه معمولی به پروژه اضافه می‌شود.

مشکلی که با استفاده از این تعریف ایجاد می‌شود، از بین رفتن کلیدواژه نوع خطی در بخش پیش‌پردازش سامانه تحلیل نوع خطی است. درنتیجه، نیاز است تا یک تعریف اختصاصی از این ماکرو نیز وجود داشته باشد و درهنگام تحلیل و بررسی قوانین مربوط به نوع خطی، از آن استفاده شود. تعریف اختصاصی ماکروی نوع خطی اجازه می‌دهد تا با جایگزینی آن، استفاده‌های نوع خطی در تحلیل از بین نرفته و حفظ شود. همچنین، کاربر متوجه این تغییر تعریف در بخش تحلیل نشده و پیچیدگی‌ای در سمت کاربر ایجاد نخواهد شد. این سازوکار از پیچیدگی کمی در سمت کاربر برخوردار بوده و به‌عنوان روش ارائه نوع خطی مناسب است. تعریف اختصاصی ماکروی نوع خطی می‌تواند مانند تعریف آورده شده در شکل ۲.۵ باشد.

^۱ library

^۲ annotate

^۳ macro

```
azhdaha.h

#define LINEAR_TYPE linear_type
```

شکل ۲.۵: تعریف اختصاصی ماکروی نوع خطی

کتابخانه‌های فعلی نوشته شده با استفاده از زبان برنامه‌نویسی C، از نوع خطی استفاده نمی‌کنند. در نتیجه راهی برای متوجه شدن از رعایت قوانین مربوط به نوع خطی در آن‌ها وجود ندارد. پس نیاز به سازوکاری است تا نسخه حاشیه‌نویسی شده سربرگ^۴ کتابخانه‌های پر استفاده این زبان تهیه شود. همچنین، نیاز است تا خود کتابخانه استاندارد این زبان نیز حاشیه‌نویسی شده و در اختیار کاربران قرار بگیرد. این حاشیه‌نویسی باید توابع مربوط به تخصیص^۵ و آزادسازی حافظه را شامل شده و جوانب مربوط به این موارد را مدنظر قرار دهد.

از آنجایی که تعداد کتابخانه‌های نوشته شده با استفاده از زبان برنامه‌نویسی C خیلی زیاد هستند، امکان دست بردن به همه آن‌ها و اضافه کردن نوع خطی به کد همه آن‌ها وجود ندارد. برای حل این چالش، یک مخزن عمومی^۶ می‌توان ایجاد نمود تا کاربران بتوانند کتابخانه‌هایی که استفاده می‌کنند را خودشان حاشیه‌نویسی کرده و با ارائه درخواست در آن قرار دهند. بدین‌صورت با کمک‌گرفتن از کاربران ابزار، می‌توان به‌طور تدریجی این مخزن را توسعه داده تا در نهایت نسخه حاشیه‌نویسی شده برای کتابخانه‌های پر استفاده این زبان، در این مخزن قرار بگیرد. همچنین، این امکان فراهم می‌شود تا این حاشیه‌نویسی‌ها به‌صورت رسمی و توسط هزاران کاربر بررسی شده تا ایرادی به قصد یا به عمد در آن‌ها وجود نداشته باشد. شکل ۳.۵ یک نمونه از حاشیه‌نویسی ممکن برای بخشی از کتابخانه استاندارد زبان برنامه‌نویسی C را نشان می‌دهد.

```
stdlib.h

/* Allocate SIZE bytes of memory. */
extern linear_type void *malloc (size_t __size);

/* Free a block allocated by 'malloc', 'realloc' or 'calloc'. */
extern void free (linear_type void *__ptr);
```

شکل ۳.۵: نمونه حاشیه‌نویسی کتابخانه استاندارد

^۴ header

^۵ allocation

^۶ public repository

پروژه‌هایی که با استفاده از زبان برنامه‌نویسی C توسعه داده شده‌اند، به دلیل عدم برخورداری این زبان از یک سامانه ساخت^۷ یکتا، ممکن است ساختارهای متنوع و پیچیده‌ای داشته باشند، در نتیجه امکان انجام تحلیل تنها با استفاده از مسیر پروژه وجود ندارد. همچنین، نمی‌توان از تمامی سامانه‌های ساخت به طور کامل پشتیبانی نمود زیرا بسیار وقت‌گیر بوده و پیچیدگی را افزایش می‌دهد. از این رو نیاز است تا یک قالب استاندارد مانند فایل دستورهای کامپایل^۸ پروژه در اختیار ابزار قرار داده شده و مسیر آن مشخص شود.

فایل دستورهای کامپایل پروژه، یک قالب استاندارد حاوی اطلاعات جامع در خصوص ساخت پروژه است که هر ورودی آن شامل مسیر مبدا فایل، دستورها، مسیرهای جست‌وجوی سربرگ‌ها و سایر پارامترهای مورد نیاز برای کامپایل آن واحد است. با دریافت این فایل می‌توان به راحتی ساختار پروژه را استخراج کرده و بدون وابستگی به یک سامانه ساخت خاص مانند Cmake یا Makefile، تحلیل را انجام داد. شکل ۴.۵ نمونه‌ای از ساختار فایل دستورهای کامپایل یک پروژه ساده را نشان می‌دهد. قابل ذکر است که این فایل توسط اکثر سامانه‌های ساخت قابل تولید بوده و حتی ابزارهای مستقلی مانند bear نیز برای این کار وجود دارند. استفاده از این قالب استاندارد، دارای مزایای زیر است:

- انطباق کامل با تنظیم‌های واقعی کامپایلر برای برنامه به منظور تحلیل دقیق‌تر
- عدم وابستگی به یک سامانه ساخت خاص
- پشتیبانی از ساختارهای پروژه پیچیده و یا ناهمگون

```
compile_commands.json

[
  {
    "arguments": [
      "/usr/bin/gcc",
      "-c",
      "main.c"
    ],
    "directory": "/root/",
    "file": "/root/main.c"
  },
]
```

شکل ۴.۵: نمونه ساختار فایل دستورهای کامپایل

^۷ build system

^۸ compile_commands.json

قبل از شروع به تجزیه کد ورودی و تحلیل آن، نیاز است تا یک مرحله پیش‌پردازش بر روی کد صورت بگیرد. هدف از این پیش‌پردازش ساده‌تر کردن کد، گسترش ماکروها^۹ و جایگذاری کتابخانه‌ها با نسخه حاشیه‌نویسی شده آن‌ها است. این پیش‌پردازش باعث می‌شود تا کد ورودی به صورت یک نسخه کامل، استاندارد و قابل تحلیل در آمده به طوری که بخش‌های بعدی بتوانند بدون پیچیدگی‌های اضافی عملیات خود را انجام دهند. ماکروهایی که در زبان برنامه‌نویسی C وجود دارند، در حالت معمول در زمان کامپایل برنامه گسترش یافته و با تعریف آن جایگزین می‌شوند. در این گسترش، کلیدواژه `include` که برای استفاده از کتابخانه‌ها کاربرد دارد نیز باز شده و با محتوای اصلی جایگزین می‌شود.

به منظور تسهیل تحلیل کد و قرارگیری تمام کد در یک مکان، در بخش پیش‌پردازش می‌توان ابتدا ماکروها را با اضافه کردن پرچم `'-E'` به مجموعه دستورهای ذخیره شده در فایل دستورهای کامپایل پروژه گسترش داده و در ادامه تحلیل را بر روی نتیجه این دستور انجام داد. قابل ذکر است که برای کامپایلرهای مختلف، این پرچم به منظور انجام پیش‌پردازش بر روی کد وجود داشته و مشکلی از بابت کامپایلر استفاده شده توسط کاربر ایجاد نخواهد شد. شکل‌های ۵.۵ نمونه‌ای از استفاده از ماکروها در زبان برنامه‌نویسی C را نشان داده و شکل ۶.۵ نتیجه انجام عملیات پیش‌پردازش بر روی این قطعه کد را نشان می‌دهد.

باتوجه به شکل‌های ۵.۴ و ۶.۴ پس از اعمال عملیات پیش‌پردازش، استفاده ماکروی `SUM` با تعریفش جایگزین شده و به جای کلیدواژه `include` نیز محتوای سربرگ `foo.h` قرار گرفته است. بدین صورت تمام کد در یک مکان قرار گرفته و تحلیل ساده‌تر می‌شود.

<pre>foo.h #define SUM(a, b) ((a) + (b)) int multiply(int a, int b);</pre>	<pre>main.c #include "foo.h" int main() { int a = SUM(1, 2); int b = multiply(2, 2); return 0; }</pre>
--	--

شکل ۵.۵: نمونه استفاده از ماکروها

^۹ macro expansion

^{۱۰} flag

```
main.c

int multiply(int a, int b);

int main() {
    int a = ((1) + (2));
    int b = multiply(2, 2);
    return 0;
}
```

شکل ۵.۶: نتیجه گسترش ماکروها در بخش پیش‌پردازش

به‌منظور انجام تحلیلی کامل و دقیق، نیاز است تا کتابخانه‌های حاشیه‌نویسی شده جایگزین کتابخانه‌های فعلی استفاده شده در پروژه شوند. این جایگزینی باید به‌گونه‌ای باشد که کاربر متوجه آن نشده و تغییری در کد و یا کتابخانه‌های استفاده شده صورت نگیرد. چنین جایگزینی‌ای با استفاده از پرچم "-I" ممکن است. این پرچم مشخص می‌کند که کامپایلر در چه مسیرهایی باید به دنبال سربرگ‌های استفاده شده در کد بگردد. قابل ذکر است که رفتار کامپایلرها به‌گونه است که اگر چندین مسیر برای یک سربرگ وجود داشته باشد، مسیری که زودتر ورودی داده شده، استفاده خواهد شد. پس اگر مسیر کتابخانه‌های حاشیه‌نویسی شده در ابتدای دستور پیش‌پردازش با استفاده از این پرچم ورودی داده شود، از آن‌ها بجای کتابخانه‌های اصلی استفاده خواهد شد. بدین‌صورت کتابخانه‌های استفاده شده با نسخه حاشیه‌نویسی شده آن‌ها جایگزین شده و تغییری در فایل‌های اصلی کتابخانه‌ها صورت نمی‌گیرد. همان‌طور که در بخش‌های قبلی توضیح داده شد نیز، این کتابخانه‌های حاشیه‌نویسی شده را می‌توان در یک مخزن عمومی قرار داده تا به راحتی قابل دسترسی باشند.

در برخی مواقع ممکن است تا نیاز باشد که حافظه تخصیص یافته با استفاده از یک تابع خاص آزاد شود. به‌طور مثال، اگر یک فایل ساخته شده باشد آنگاه برای بستن و آزادسازی حافظه بافر آن باید از تابع مخصوص بستن فایل‌ها استفاده شود. نوع خطی اگرچه می‌تواند تضمین کند که این فایل بسته می‌شود، ولی نمی‌تواند تضمین کند که این فایل با استفاده از چه تابعی بسته می‌شود. برای رفع این چالش، می‌توان نوع خطی را به‌همراه یک برچسب^{۱۱} تعریف کرده تا دو نوع خطی با برچسب‌های متفاوت، بجای یکدیگر استفاده نشوند. همچنین، به‌منظور سازگاری با نوع خطی فعلی، می‌توان نوع خطی بدون برچسب را نیز همچنان پشتیبانی نمود.

نوع خطی فقط برای تخصیص و آزادسازی حافظه کاربرد نداشته و می‌توان عدد صحیح خطی نیز داشت. کاربرد چنین نوعی می‌تواند در زمان بازکردن و بستن سوکت‌ها^{۱۲} باشد زیرا با باز کردن یک سوکت، یک توصیف‌گر فایل^{۱۳} بازگردانده

^{۱۱} label

^{۱۲} socket

^{۱۳} file descriptor

می‌شود که یک عدد صحیح است. تعریف این توصیف‌گر فایل به عنوان نوع خطی می‌تواند تضمین کند که این سوکت درآینده بسته خواهد شد، زیرا این عدد صحیح باید مصرف شود و باتوجه به حاشیه‌نویسی‌ها، تنها تابعی که آن را مصرف می‌کند، تابع مربوط به بستن سوکت‌ها است.

از آنجایی که بررسی یک تابع برای برقراری قوانین نوع خطی مستقل از بررسی دیگر توابع است، می‌توان این عملیات را به‌صورت موازی و به‌صورت چند هسته‌ای انجام داده و در نهایت گزارش‌های به دست‌آمده را در خروجی نمایش داد. همچنین، به‌منظور افزایش عملکرد و سرعت ابزار، برای پیاده‌سازی الگوریتم جست‌وجوی عمق اول بهتر است از پشته^{۱۴} استفاده شده و از توابع بازگشتی اجتناب شود. گزارشی که به کاربر داده می‌شود نیز باید تا جای ممکن کامل بوده تا مشکلی از بابت پیدا کردن علت خطا و مکان آن وجود نداشته باشد. بدین‌منظور در زمان پیمایش گراف کنترل جریان کد می‌توان روندی که یک متغیر دارای نوع خطی تا زمان رخداد خطا طی می‌کند را ثبت کرده و در نهایت این روند را درکنار خطا نمایش داد.

با به کارگیری پیشنهادهای داده شده، ابزار می‌تواند در پروژه‌های بزرگ‌تر و پیچیده‌تر به کار گرفته شده و نحو زبان برنامه‌نویسی C را به‌طور کامل پشتیبانی کند. درنتیجه، توسعه‌دهندگان قادر خواهند بود تا از آن برای تحلیل کد در مقیاس سازمانی و شناسایی الگوهای پیچیده نشت حافظه بهره ببرند. همچنین، این ابزار می‌تواند با فراهم کردن رابط‌هایی برای اتصال به محیط‌های یکپارچه توسعه^{۱۵}، فرآیند بازبینی و اصلاح کد را تسهیل کند. در نهایت این ابزار می‌تواند به‌عنوان پایه‌ای برای پژوهش‌های آتی در زمینه اثبات‌های صوری و تضمین ویژگی‌های ایمنی حافظه در پروژه‌های نرم‌افزاری حساس نیز مورد استفاده قرار بگیرد.

^{۱۴} stack

^{۱۵} IDE - integrated development environment

فهرست مراجع

- [۱] L. T. C. Melo, R. G. Ribeiro, M. R. de Araujo, and F. M. Q. Pereira, "Type inference for C: Applications to the static analysis of incomplete programs," *ACM Trans. Program. Lang. Syst.*, vol. 42, no. 4, pp. 1-34, Jul. 2020. doi: 10.1145/3421472.
- [۲] N. S. Papaspyrou, *A Formal Semantics for the C Programming Language*, Ph.D. dissertation, National Technical University of Athens, Athens, Greece, Feb. 1998.
- [۳] H. Aslanyan, Z. Gevorgyan, R. Mkoyan, H. Movsisyan, S. Sargsyan, "Static analysis methods for memory leak detection: A survey," in *Proc. 2022 Ivannikov Memorial Workshop (IVMEM)*, Yerevan, Armenia, Sep. 2022, pp. 1-6. doi: 10.1109/IVMEM57067.2022.9983955.
- [۴] B. C. Pierce, *Types and Programming Languages*. Cambridge, MA, USA: The MIT Press, 2002.
- [۵] X. Rival and K. Yi, *Introduction to Static Analysis: An Abstract Interpretation Perspective*. Cambridge, MA, USA: MIT Press, 2020.
- [۶] N. Matsakis, "Must move types," Baby-Steps, 16 Mar. 2023. [Online]. Available: <https://smallcultfollowing.com/babysteps/blog/2023/03/16/must-move-types/>
- [۷] A. Ghadiri, "Azhdaha: A compiler frontend for C programming language to detect memory issues," GitHub, 2025. [Online]. Available: <https://github.com/skye-tan/azhdaha/>